VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Komponenta výukového serveru TI - Amortizovaná složitost 1

Component of Teaching Server for Theoretical Computer Science - Amortized Complexity 1

Martin Dressler

Bakalářská práce

Vedoucí práce: Ing. Martin Kot, Ph.D.

Ostrava, 2026

VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Č.j.VSB/25/116928
vsboee25037F9C

Zadání bakalářské práce

Student: **Martin Dressler**

Studijní program: B0613A140014 Informatika

Téma: Komponenta výukového serveru TI - Amortizovaná složitost 1

Component of Teaching Server for Theoretical Computer Science - Amortized Complexity 1

Jazyk vypracování: čeština

Zásady pro vypracování:

V rámci diplomových a bakalářských prací vzniká výukový server pro předměty teoretické informatiky. Jedná se o sadu dynamických webových stránek umožňujících studentům pochopení různých typů úloh a problémů. Na rozdíl od běžných výukových textů s pevně daným počtem ukázkových příkladů umí tyto stránky generovat libovolně mnoho ukázek na základě vstupů od uživatele. Cílem této konkrétní bakalářské práce je vytvořit komponentu pro pochopení počítání složitosti algoritmů s využitím tzv. amortizované složitosti.

Vytvořte dynamické webové stránky umožňující uživateli následující:
1. Zobrazit si vhodným způsobem simulaci běhu alespoň 3 různých algoritmů, u kterých je vhodné složitost analyzovat pomocí amortizované složitosti.
2. Vstupy těchto algoritmů bude moci uživatel zadávat třemi způsoby:
a) Vhodným, uživatelsky přívětivým, způsobem ručně.
b) Nechat si vstup vygenerovat zcela náhodně podle nastavených parametrů.
c) Nechat si vygenerovat různě velké ukázkové vstupy (typicky např. nejhorší případ, nejlepší případ apod.).
3. Při simulaci algoritmů se bude uživateli zobrazovat:
a) Informace vystihující aktuální stav výpočtu algoritmu (např. obsahy různých datových struktur atd.).
b) Počet provedených kroků.
c) Aktuální hodnotu potenciálu (při použití tzv. potenciálové metody) nebo naspořené mince (při použití tzv. účetní metody).
4. Ke každému algoritmu bude přímo na vytvořené stránce také k zobrazení samotné odvození složitosti tohoto algoritmu (např. formou statické stránky).

Studenti řešící toto zadání s rozdílným číslem v názvu mohou (ale nemusí) spolupracovat tak, že výsledek může mít společné uživatelské rozhraní apod. Ale každý bude implementovat jiné 3 algoritmy.

Seznam doporučené odborné literatury:

[1] Tarjan, Robert Endre: "Amortized Computational Complexity", SIAM Journal on Algebraic and Discrete Methods, 6 (2): 306–318, doi:10.1137/0606031
[2] Sawa, Zdeněk: "Teoretická informatika", podklady pro přednášky, VŠB - Technická univerzita Ostrava, dostupné z URL: https://www.cs.vsb.cz/sawa/ti/slides/ti-slides-03.pdf

[3] Kozen, Dexter: "CS 3110 Lecture 20: Amortized Analysis", Cornell University, dostupné z URL: http://www.cs.cornell.edu/courses/cs3110/2011sp/lectures/lec20-amortized/amortized.htm

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Martin Kot, Ph.D.**

Datum zadání: 01.09.2024

Datum odevzdání: 30.04.2026

Garant studijního programu: Ing. Tomáš Fabián, Ph.D.

V IS EDISON zadáno: 18.08.2025 09:41:44

Abstrakt

Tato bakalářská práce se zabývá problematikou amortizované složitosti. Cílem praktické části bylo vytvořit komponentu výukového serveru pro předmět Teoretická informatika v navazujícím magisterském studiu. Výsledná aplikace má podobu dynamické webové stránky, která vizualizuje vybrané algoritmy vhodné pro analýzu amortizované složitosti a umožňuje uživateli pracovat s různými typy vstupů. Součástí aplikace je zobrazení jednotlivých kroků algoritmů, změn stavů datových struktur a dalších hodnot důležitých pro pochopení jejich chování. Teoretická část se věnuje vysvětlení amortizované složitosti, souvisejících pojmů a metod, popisu implementace vytvořené komponenty a charakteristice vybraných algoritmů. Výsledkem práce je výukový nástroj, který může přispět k lepšímu pochopení amortizované složitosti studenty.

Klíčová slova

Amortizovaná složitost, potenciální metoda, vizualizace algoritmů, výukový server, teoretická informatika, webová aplikace, HTML5, CSS3, JavaScript, Bootstrap

Abstract

This bachelor's thesis deals with the issue of amortized complexity. The goal of the practical part was to create a component of a teaching server for the course Theoretical Computer Science in the follow-up master's degree programme. The resulting application takes the form of a dynamic web page that visualizes selected algorithms suitable for amortized complexity analysis and allows the user to work with different types of inputs. The application includes the display of individual algorithm steps, changes in the states of data structures, and other values important for understanding their behavior. The theoretical part focuses on the explanation of amortized complexity, related concepts and methods, the description of the implementation of the created component, and the characteristics of the selected algorithms. The result of the thesis is an educational tool that can contribute to a better understanding of amortized complexity by students.

Keywords

Amortized complexity, potential method, algorithm visualization, teaching server, theoretical computer science, web application, HTML5, CSS3, JavaScript, Bootstrap

Poděkování

Rád bych poděkoval vedoucímu mé bakalářské práce Ing. Martinu Kotovi, Ph.D. za odborné vedení, cenné rady, trpělivost a čas, který mi věnoval při zpracování této práce. Zároveň mu děkuji za možnost věnovat se tématu, které může přispět k lepšímu pochopení amortizované složitosti budoucím studentům.

Obsah

Seznam použitých symbolů a zkratek **8**

Seznam obrázků **9**

Seznam tabulek **10**

1 Úvod **11**

2 Základní problematika amortizované složitosti **13**

2.1 Základní pojmy analýzy algoritmů . . . 13

2.1.1 Vymezení základních pojmů . . . 13

2.1.2 Přehled základních charakteristik algoritmů . . . 14

2.1.3 Návaznost na amortizovanou analýzu . . . 14

2.2 Časová složitost algoritmů . . . 15

2.2.1 Velikost vstupu a definice časové složitosti . . . 15

2.2.2 Časová složitost v nejhorším a průměrném případě . . . 16

2.2.3 Asymptotická notace a třídy růstu funkcí . . . 18

2.2.4 Praktický význam časové složitosti . . . 19

2.3 Amortizovaná složitost a její význam . . . 19

2.3.1 Vymezení amortizované složitosti . . . 19

2.3.2 Význam amortizované analýzy . . . 20

2.3.3 Srovnání s ostatními přístupy . . . 20

2.4 Metody amortizované analýzy . . . 21

2.4.1 Agregovaná metoda . . . 22

2.4.2 Účetní metoda . . . 22

2.4.3 Potenciálová metoda . . . 22

2.4.4 Vztah mezi jednotlivými metodami . . . 22

2.5 Potenciálová metoda . . . 23

2.5.1 Princip potenciálové metody . . . 23

2.5.2 Potenciálová funkce a amortizovaná cena operace . . . 24

2.5.3 Význam a využití potenciálové metody . . . 24
2.6 Vybrané algoritmy vhodné pro amortizovanou analýzu . . . 25
2.6.1 Algoritmy vhodné pro amortizovanou analýzu . . . 25
2.6.2 Algoritmy vybrané do praktické části práce . . . 25
2.6.3 Přehled vybraných algoritmů . . . 26

3 Návrh komponenty výukového serveru 27
3.1 Cíl a role navrhované komponenty . . . 27
3.1.1 Zařazení komponenty do výukového serveru . . . 28
3.1.2 Hlavní účel komponenty . . . 28
3.1.3 Přínos komponenty pro výuku amortizované složitosti . . . 29
3.2 Požadavky na komponentu . . . 29
3.2.1 Funkční požadavky . . . 29
3.2.2 Požadavky na uživatelskou interakci . . . 31
3.2.3 Požadavky na názornost a výukovou hodnotu . . . 31
3.3 Výběr algoritmů pro praktickou část . . . 31
3.3.1 Kritéria výběru algoritmů . . . 31
3.3.2 Multipop na zásobníku jako jednoduchý příklad . . . 32
3.3.3 Fronta využívající dva zásobníky jako středně obtížný příklad . . . 32
3.3.4 Splay strom jako pokročilejší příklad . . . 33
3.4 Návrh funkcionality komponenty . . . 33
3.4.1 Režimy zadávání vstupů . . . 33
3.4.2 Zobrazení průběhu algoritmu po jednotlivých krocích . . . 35
3.4.3 Zobrazení počtu kroků a potenciálu . . . 37
3.4.4 Zobrazení doplňujících informací o aktuálním stavu algoritmu . . . 38
3.5 Návrh uživatelského rozhraní . . . 39
3.5.1 Návrh hlavní stránky a výběru algoritmu . . . 39
3.5.2 Návrh ovládacích prvků simulace . . . 40
3.5.3 Návrh vizualizace datových struktur . . . 41
3.5.4 Návrh informačních panelů a doprovodných vysvětlení o algoritmech . . . 43
3.6 Návrh struktury komponenty . . . 44
3.6.1 Rozdělení komponenty na společnou a algoritmicky specifickou část . . . 44
3.6.2 Návaznost jednotlivých částí komponenty . . . 44
3.6.3 Možnosti budoucího rozšíření . . . 44

4 Implementace komponenty 45

5 Závěr 46

Seznam použitých zkratek a symbolů

TI	–	Teoretická informatika
API	–	Application Programming Interface
UI	–	User Interface
DOM	–	Document Object Model
HTML	–	HyperText Markup Language
CSS	–	Cascading Style Sheets
JS	–	JavaScript
LIFO	–	Last In, First Out
FIFO	–	First In, First Out
BST	–	Binary Search Tree
PDF	–	Portable Document Format
URL	–	Uniform Resource Locator
HTTP	–	HyperText Transfer Protocol

Seznam obrázků

2.2.1 Časová složitost v nejhorším případě podle [1] 16
2.2.2 Časová složitost v průměrném případě podle [1] 17

3.4.1 Hlavní nabídka režimů zadávání vstupů u zvoleného algoritmu 34
3.4.2 Příklady jednotlivých režimů zadávání vstupů 35
3.4.3 Zobrazení průběhu algoritmu po jednotlivých krocích v režimu syntaxe 36
3.4.4 Automatický průběh simulace v režimu nejlepšího/nejhoršího případu 36
3.4.5 Zobrazení počtu kroků a aktuální hodnoty potenciálu v průběhu simulace 37
3.4.6 Detailní doplňující informace dostupné v historii provedených kroků 38
3.4.7 Doplňující informační panel zobrazující aktuální stav operace při práci s algoritmem 39
3.5.1 Hlavní stránka komponenty s výběrem algoritmů, přepínačem jazykových mutací a tlačítkem pro informace o bakalářské práci 40
3.5.2 Příklad ovládacích prvků simulace v režimu syntaxe 41
3.5.3 Příklady vizualizace datových struktur použitých v navrhované komponentě 42
3.5.4 Příklad informačního panelu s doprovodným vysvětlením vybraného algoritmu . . . 43

Seznam tabulek

2.1.1 Základní pojmy používané při analýze algoritmů 14
2.2.1 Příklad růstu doby výpočtu pro vybrané třídy časové složitosti podle [1] 18
2.3.1 Srovnání vybraných přístupů k hodnocení složitosti 21
2.4.1 Základní metody amortizované analýzy 23
2.6.1 Vybrané algoritmy vhodné pro amortizovanou analýzu 26

3.2.1 Hlavní funkční požadavky navrhované komponenty 30

Kapitola 1

Úvod

Hlavním cílem této bakalářské práce bylo vytvoření komponenty pro výukový server v oblasti teoretické informatiky. V praktické části bylo potřeba navrhnout a implementovat dynamickou webovou aplikaci, která by sloužila jako názorný a interaktivní nástroj pro pochopení vybraných algoritmů a principů amortizované složitosti. Výsledná aplikace má studentům umožnit sledovat průběh jednotlivých operací, změny stavů datových struktur a další důležité hodnoty potřebné pro lepší pochopení chování algoritmů i jejich amortizované složitosti.

Problematika amortizované složitosti patří mezi velmi důležitá témata při studiu algoritmů a datových struktur. Její význam spočívá v tom, že umožňuje lépe popsat chování algoritmů v situacích, kdy jednotlivé operace nemají vždy stejnou cenu a jejich skutečná náročnost se projeví až v rámci delší posloupnosti kroků. U některých algoritmů a datových struktur nestačí hodnotit jednotlivé operace samostatně, protože jejich skutečná efektivita se ukáže teprve při sledování delší posloupnosti operací.

I když je amortizovaná složitost významnou součástí TI, bývá pro mnoho studentů obtížně pochopitelná. Hlavním důvodem je její abstraktnější charakter, protože při snaze o její pochopení nestačí sledovat pouze výsledek jedné operace, ale je také nutné porozumět souvislostem mezi více operacemi, změnám stavů datových struktur a principům, podle nichž se náklady mezi jednotlivými kroky rozkládají. Z tohoto důvodu byla tato bakalářská práce vytvořena jako podpora teoretického výkladu prostřednictvím interaktivní vizualizace a možnosti sledovat chování algoritmů na různých příkladech.

Výsledkem této práce je dynamická webová aplikace, která může sloužit budoucím studentům jako výuková pomůcka pro názornou vizualizaci vybraných algoritmů. Důraz je kladen především na možnost aktivní práce uživatele se vstupy a na sledování chování algoritmů v jednotlivých krocích. Student tak může lépe porozumět tomu, jak se mění stav datové struktury, jak se vyvíjí průběh operací a proč nelze u některých algoritmů posuzovat jejich efektivitu pouze na základě jedné samostatné operace. Výsledná komponenta má za cíl propojit teoretický výklad s praktickou ukázkou a přispět k názornějšímu pochopení celé problematiky.

Tato práce je rozdělena do několika hlavních kapitol. Po úvodní kapitole následuje kapitola věnovaná základní problematice amortizované složitosti, ve které jsou vysvětleny klíčové pojmy a principy související s touto oblastí. Další kapitola se zaměřuje na návrh komponenty výukového serveru a na popis jejích hlavních vlastností a požadavků. Následující kapitola se věnuje samotné implementaci aplikace a použitým technologiím. Závěrečná kapitola shrnuje dosažené výsledky a zhodnocuje přínos vytvořeného řešení.

Kapitola 2

Základní problematika amortizované složitosti

Druhá kapitola se věnuje základní problematice amortizované složitosti a pojmům, které s ní úzce souvisejí. Nejprve shrnuje obecné principy analýzy algoritmů a pojem časové složitosti, které tvoří východisko pro pochopení amortizované analýzy. Následně vysvětluje význam amortizované složitosti a ukazuje, proč není vždy vhodné hodnotit efektivitu algoritmu pouze podle nákladů jedné samostatné operace. Dále představuje základní metody amortizované analýzy se zvláštním důrazem na potenciálovou metodu. V závěru kapitoly je zařazen také stručný přehled algoritmů, u nichž lze amortizovanou analýzu využít, včetně algoritmů vybraných do praktické části této práce.

2.1 Základní pojmy analýzy algoritmů

V této části jsou stručně vymezeny základní pojmy analýzy algoritmů a naznačena jejich návaznost na amortizovanou analýzu.

2.1.1 Vymezení základních pojmů

Analýza algoritmů spočívá ve zkoumání vlastností algoritmů, především jejich správnosti a náročnosti výpočtu. V běžné praxi bývá přesné vyjádření složitosti obtížné, protože závisí na konkrétním výpočetním modelu i na konkrétní implementaci algoritmu. Z tohoto důvodu se v teorii algoritmů využívá asymptotické vyjadřování, které potlačuje méně podstatné detaily a umožňuje soustředit se především na to, jak rychle roste náročnost algoritmu se zvětšující se velikostí vstupu [1].

Mezi nejzákladnější pojmy, které se v analýze algoritmů běžně používají, patří algoritmus, vstup, výstup, korektnost a složitost. Algoritmus lze chápat jako přesně definovaný postup, který vede od vstupních dat k požadovanému výsledku. Vstup představuje data, nad nimiž je algoritmus vykonáván, zatímco výstup je výsledkem jejich zpracování. Korektnost vyjadřuje, zda algoritmus pro

přípustný vstup poskytuje správný výstup, zatímco složitost popisuje, jaké množství prostředků je potřeba k dosažení výsledku. Teprve kombinace těchto hledisek umožňuje smysluplně hodnotit kvalitu navrženého řešení.

2.1.2 Přehled základních charakteristik algoritmů

Pro lepší přehlednost shrnuje tabulka 2.1.1 vybrané základní pojmy, se kterými se v rámci analýzy algoritmů dále pracuje.

Tabulka 2.1.1: Základní pojmy používané při analýze algoritmů

Pojem	**Stručné vysvětlení**
Algoritmus	Jednoznačně definovaný postup, který převádí vstupní data na požadovaný výstup.
Vstup	Data, nad nimiž je algoritmus vykonáván. Velikost vstupu obvykle ovlivňuje náročnost výpočtu.
Výstup	Výsledek zpracování vstupu algoritmem.
Korektnost	Vlastnost algoritmu, která určuje, zda algoritmus poskytuje správný výsledek pro přípustné vstupy.
Časová složitost	Vyjadřuje náročnost algoritmu z hlediska času, resp. počtu provedených operací.
Paměťová složitost	Vyjadřuje nároky algoritmu na paměť během výpočtu.
Asymptotické vyjádření	Způsob popisu růstu náročnosti algoritmu pro velké vstupy bez zdůrazňování méně podstatných detailů.

2.1.3 Návaznost na amortizovanou analýzu

Pro potřeby této práce je podstatné, že u některých datových struktur a algoritmů nemusí samostatné hodnocení jedné operace dostatečně vystihnout skutečné chování celého řešení. V takových situacích je vhodné sledovat náklady celé posloupnosti operací, což je princip, na němž je založena amortizovaná analýza [2]. Tento přístup je podrobněji vysvětlen v následujících částech kapitoly.

2.2 Časová složitost algoritmů

V této části je nejprve vysvětlen pojem velikosti vstupu a základní způsob vyjádření časové složitosti algoritmu. Následně je pozornost věnována časové složitosti v nejhorším a průměrném případě, asymptotické notaci a vybraným třídám růstu funkcí. Závěr této části ukazuje praktický význam časové složitosti při porovnávání algoritmů a její vztah k amortizované analýze.

2.2.1 Velikost vstupu a definice časové složitosti

Časová složitost algoritmů patří mezi nejdůležitější charakteristiky používané při hodnocení efektivity algoritmických řešení. Počítače sice pracují velmi rychle, jejich výpočetní možnosti však nejsou neomezené, a proto i provedení jednotlivých instrukcí vyžaduje určitý čas. Stejný problém navíc může být řešen více různými algoritmy, jejichž doba výpočtu se může výrazně lišit. Z tohoto důvodu je vhodné algoritmy mezi sebou porovnávat a zkoumat jejich náročnost z hlediska počtu provedených operací [1].

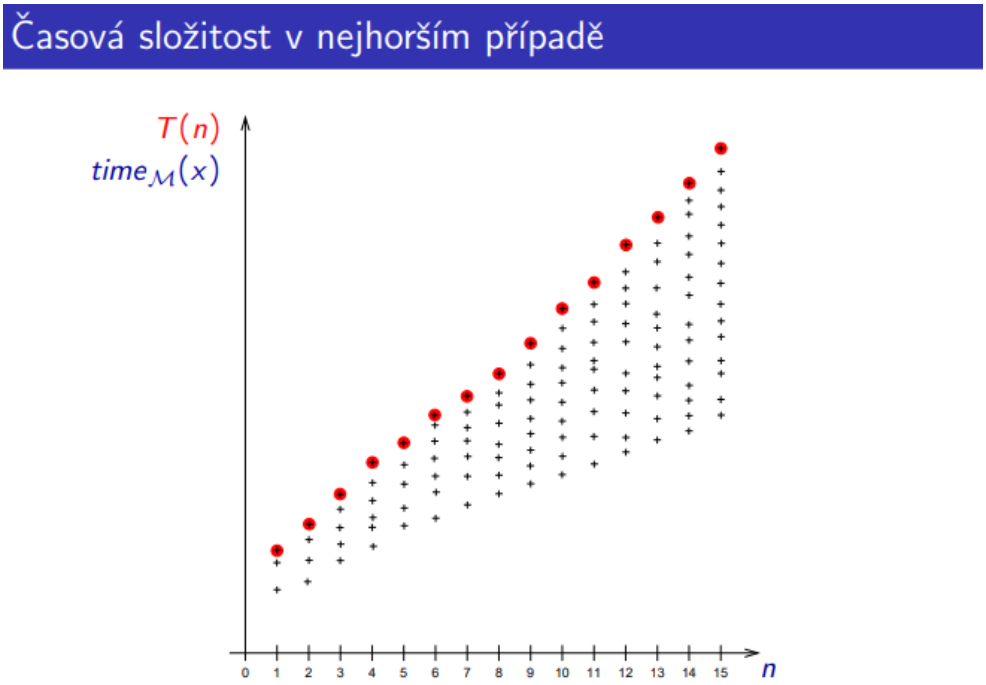
Při běžném programování je sice možné algoritmus implementovat a následně změřit dobu jeho běhu na konkrétních vstupních datech, takový přístup však neposkytuje dostatečně obecnou informaci o chování algoritmu pro všechny možné vstupy. V teorii algoritmů se proto časová složitost nevyjadřuje přímo pomocí skutečného času naměřeného na konkrétním zařízení, ale spíše pomocí počtu provedených instrukcí nebo elementárních operací. Díky tomu je možné odhlédnout od konkrétního hardwaru a zaměřit se na samotné chování algoritmu [1].

Důležitým pojmem při analýze časové složitosti je velikost vstupu. Pro různé typy problémů může být definována různě. Například u problému třídění lze za velikost vstupu považovat počet prvků ve vstupní posloupnosti, u problémů pracujících s čísly může být přirozenější zvolit počet bitů potřebných k jejich zápisu a u grafových problémů lze velikost vstupu vyjádřit například dvojicí hodnot udávajících počet vrcholů a počet hran. Zvolená definice velikosti vstupu tedy není vždy pevně dána, ale měla by přirozeně odpovídat povaze analyzovaného problému [1].

Uvažujme nyní konkrétní algoritmus a jeho implementaci. Pro vstupy stejné velikosti nemusí algoritmus vždy vykonat stejný počet instrukcí, protože jeho chování může záviset na konkrétním uspořádání vstupních dat. Proto se v teorii často zavádí funkce $T(n)$, která pro každou velikost vstupu n udává maximální počet instrukcí provedených algoritmem na vstupu velikosti n. Takto definovaná funkce představuje časovou složitost algoritmu v nejhorším případě a umožňuje zachytit horní mez výpočetní náročnosti [1].

2.2.2 Časová složitost v nejhorším a průměrném případě

Pro lepší názornost je princip časové složitosti v nejhorším případě znázorněn na obrázku 2.2.1. Jednotlivé body představují různé vstupy stejné nebo podobné velikosti a jejich skutečnou časovou náročnost. Funkce $T(n)$ zde odpovídá horní hranici těchto hodnot, tedy nejhoršímu případu pro danou velikost vstupu.

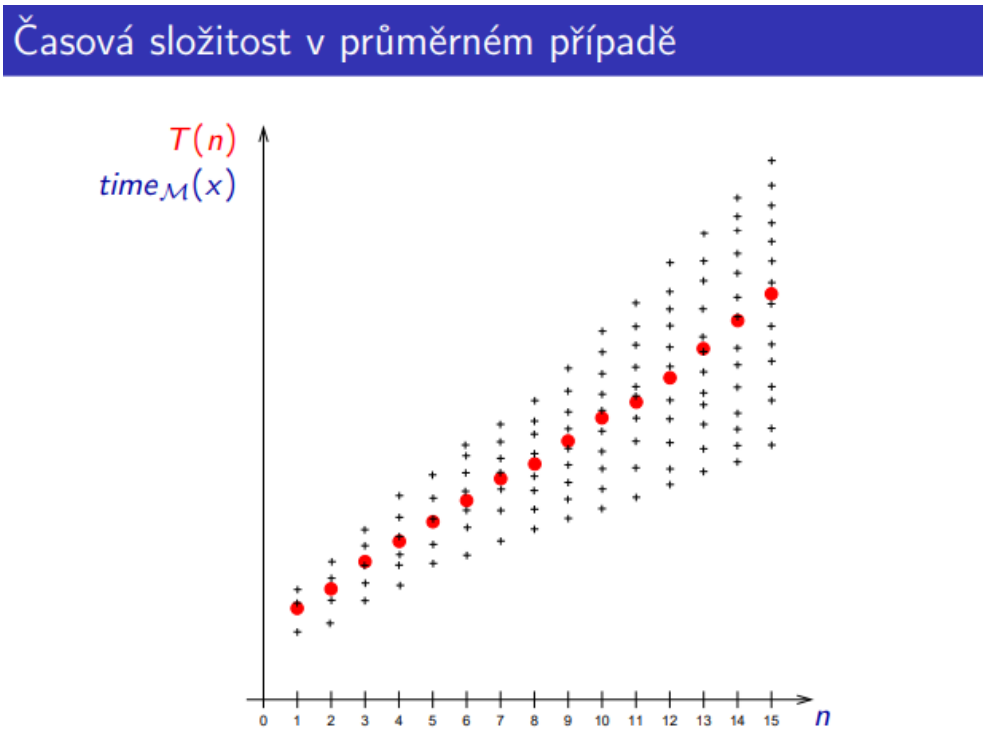


Obrázek 2.2.1: Časová složitost v nejhorším případě podle [1]

Obrázek 2.2.1 ukazuje, že pro vstupy stejné velikosti nemusí algoritmus vždy vykonat stejný počet operací. Časová složitost v nejhorším případě proto sleduje maximální hodnotu těchto nákladů a poskytuje tak bezpečný odhad horní meze časové náročnosti algoritmu.

Vedle časové složitosti v nejhorším případě má smysl zkoumat také časovou složitost v průměrném případě. V tomto případě není funkce $T(n)$ určena maximální hodnotou přes všechny vstupy dané velikosti, ale aritmetickým průměrem odpovídajících nákladů. Určení průměrného případu však bývá zpravidla obtížnější než určení nejhoršího případu, protože vyžaduje přesnější představu o množině vstupů a jejich rozdělení. Ve většině případů se proto při analýze algoritmů častěji vychází právě z nejhoršího případu [1].

Pro názornost je na obrázku 2.2.2 znázorněna časová složitost v průměrném případě. Na rozdíl od nejhoršího případu zde funkce $T(n)$ nepředstavuje horní obálku všech hodnot, ale jejich průměrné chování pro vstupy stejné velikosti.



Obrázek 2.2.2: Časová složitost v průměrném případě podle [1]

Časová složitost v nejhorším a průměrném případě se často příliš neliší, i když v některých situacích může být rozdíl významný. Zároveň se uvádí, že zkoumání složitosti v nejlepším případě obvykle nemá příliš velký praktický význam [1].

2.2.3 Asymptotická notace a třídy růstu funkcí

Přesné vyjádření časové složitosti bývá často komplikované a silně závisí na zvoleném výpočetním modelu i na detailech implementace. Ve většině případů navíc není nutné znát přesný počet instrukcí, ale stačí odhadnout, jak rychle tento počet roste se zvětšující se velikostí vstupu. Právě z tohoto důvodu se zavádí asymptotická notace, která zanedbává méně podstatné detaily a umožňuje soustředit se na dominantní složku růstu sledované funkce [1].

V rámci asymptotické notace se nejčastěji používají symboly O, Ω a Θ. Zápis $O(g(n))$ vyjadřuje horní asymptotickou mez růstu funkce, tedy skutečnost, že sledovaná funkce neroste rychleji než funkce $g(n)$ až na konstantní násobek. Naopak $\Omega(g(n))$ vyjadřuje dolní asymptotickou mez a $\Theta(g(n))$ charakterizuje situaci, kdy funkce roste stejnou rychlostí jako $g(n)$ z hlediska horní i dolní meze současně. Kromě těchto symbolů se používají také $o(g(n))$ a $\omega(g(n))$, které popisují přísně pomalejší, respektive přísně rychlejší růst [1].

Big-O notace popisuje asymptotické chování funkcí a umožňuje při analýze algoritmů zanedbat konstanty i pomaleji rostoucí členy [3]. Mezi běžné třídy patří například $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$, polynomiální růst a exponenciální růst, přičemž exponenciální funkce rostou výrazně rychleji než funkce polynomiální [3].

Na základě asymptotického chování lze algoritmy rozdělovat do několika základních tříd. Za logaritmické se považují algoritmy se složitostí $\Theta(\log n)$, za lineární algoritmy se složitostí $\Theta(n)$, za kvadratické se považují algoritmy se složitostí $\Theta(n^2)$ a podobně. Významnou skupinu tvoří také polynomiální algoritmy, tedy algoritmy, jejichž časová složitost je shora omezena nějakým polynomem. Naopak exponenciální nebo faktoriální složitosti obvykle vedou k tomu, že algoritmus je použitelný pouze pro velmi malé vstupy [1, 3].

Praktický význam rozdílné rychlosti růstu funkcí ukazuje tabulka 2.2.1. Ta vychází z předpokladu, že algoritmus zpracovává vstup velikosti n, provede $T(n)$ operací a provedení jedné operace trvá $1\,\mu s$. Je z ní patrné, že rozdíly mezi jednotlivými třídami složitosti se s rostoucí velikostí vstupu velmi rychle zvětšují [1].

Tabulka 2.2.1: Příklad růstu doby výpočtu pro vybrané třídy časové složitosti podle [1]

$T(n)$ / n	20	40	60	80	100	200	500	1000
n	20 μs	40 μs	60 μs	80 μs	0.1 ms	0.2 ms	0.5 ms	1 ms
$n \log n$	86 μs	0.213 ms	0.354 ms	0.506 ms	0.664 ms	1.528 ms	4.48 ms	9.96 ms
n^2	0.4 ms	1.6 ms	3.6 ms	6.4 ms	10 ms	40 ms	0.25 s	1 s
n^3	8 ms	64 ms	0.216 s	0.512 s	1 s	8 s	125 s	16.7 min.
n^4	0.16 s	2.56 s	12.96 s	42 s	100 s	26.6 min.	17.36 hod.	11.57 dní
2^n	1.05 s	12.75 dní	36560 let	$38.3 \cdot 10^9$ let	$40.1 \cdot 10^{15}$ let	$50 \cdot 10^{45}$ let	$10.4 \cdot 10^{136}$ let	–
$n!$	77147 let	$2.59 \cdot 10^{34}$ let	$2.64 \cdot 10^{68}$ let	$2.27 \cdot 10^{105}$ let	$2.96 \cdot 10^{144}$ let	–	–	–

Z tabulky 2.2.1 je patrné, že i relativně malé rozdíly v asymptotické třídě mohou mít při větších vstupech zásadní dopad na skutečnou dobu výpočtu. Zatímco lineární nebo logaritmické algoritmy

zůstávají použitelné i pro velké vstupy, algoritmy s exponenciální nebo faktoriální složitostí se stávají prakticky neproveditelnými velmi rychle [1, 3].

2.2.4 Praktický význam časové složitosti

Při interpretaci asymptotické notace je však nutné mít na paměti i její omezení. Asymptotické odhady vypovídají pouze o tom, jak roste náročnost s rostoucí velikostí vstupu, ale neříkají nic o skutečné době běhu na konkrétním počítači. V notaci mohou být navíc skryty velké konstanty, takže algoritmus s asymptoticky lepší složitostí nemusí být v praxi rychlejší pro malé nebo středně velké vstupy [1].

Tuto skutečnost lze ilustrovat také na třídicích algoritmech. Například quicksort má v nejhorším případě složitost $\Theta(n^2)$, zatímco heapsort $\Theta(n \log n)$, ale přesto bývá quicksort v praxi často velmi rychlý. Z toho vyplývá, že časová složitost je sice velmi důležitým nástrojem pro porovnávání algoritmů, ale při hodnocení konkrétního řešení je třeba zohlednit i další okolnosti.

Z praktického hlediska bývají za efektivní považovány především algoritmy s polynomiální časovou složitostí. Ani zde však nelze formulovat zcela absolutní závěry, protože například algoritmus se složitostí $\Theta(n^{100})$ je sice formálně polynomiální, avšak z praktického hlediska obtížně použitelný. Naopak některé algoritmy s horší teoretickou složitostí mohou být pro malé vstupy nebo specifické úlohy stále využitelné [1, 3].

Pro potřeby této práce je důležité, že klasická časová složitost obvykle hodnotí náklady jednotlivých výpočtů nebo jednotlivých operací vzhledem k velikosti vstupu. U některých datových struktur a algoritmů však může být vhodnější sledovat celkové náklady delší posloupnosti operací. Právě tímto pohledem se zabývá amortizovaná analýza, která je tématem následující části kapitoly [4, 2].

2.3 Amortizovaná složitost a její význam

V této podkapitole je nejprve vymezen pojem amortizované složitosti a vysvětlen základní princip tohoto způsobu analýzy. Následně je pozornost věnována významu amortizované analýzy při hodnocení algoritmů a datových struktur. V závěru této podkapitoly je amortizovaná analýza srovnána s dalšími přístupy používanými při posuzování náročnosti algoritmů.

2.3.1 Vymezení amortizované složitosti

Amortizovaná složitost představuje způsob hodnocení algoritmů a datových struktur, při němž se nesleduje pouze cena jedné izolované operace, ale celkové náklady delší posloupnosti operací. Tento přístup vychází z toho, že jednotlivé operace v posloupnosti nemusí mít stejnou cenu. Některé mohou být velmi levné, jiné naopak výrazně dražší, a proto by samostatné hodnocení jedné operace mohlo vést ke zkreslenému pohledu na skutečné chování algoritmu [4, 2].

Amortizovaná analýza představuje analýzu posloupnosti operací v nejhorším případě, jejímž cílem je získat těsnější odhad celkových nebo průměrných nákladů na jednu operaci, než kdyby byla každá operace analyzována samostatně [2]. Z toho vyplývá, že dražší operace nejsou posuzovány izolovaně, ale v souvislosti s ostatními operacemi v posloupnosti. Jejich vysoká cena se tak rozkládá mezi více levnějších operací, takže dlouhodobé chování algoritmu může být z hlediska jedné operace příznivější.

2.3.2 Význam amortizované analýzy

Význam amortizované analýzy spočívá především v tom, že poskytuje realističtější pohled na chování algoritmů a datových struktur v situacích, kdy se náklady jednotlivých operací výrazně liší. Amortizace je chápána jako technika „zprůměrování v čase“ a amortizovaná doba běhu představuje realistickou a zároveň robustní míru složitosti [4]. Tento přístup je užitečný zejména tam, kde se vyskytují občasné drahé operace, které jsou však v delší posloupnosti kompenzovány velkým množstvím operací levných.

Amortizovaná analýza je důležitá také proto, že se nespoléhá na pravděpodobnostní předpoklady o rozdělení vstupů. Oproti průměrnému případu není založena na odhadu, jak často se určitý typ vstupu vyskytne. Hodnocení je prováděno nad posloupností operací v nejhorším případě a může vést k přesnějším odhadům, než kdyby byly jednotlivé operace posuzovány odděleně [2]. Tato vlastnost představuje výhodu zejména u datových struktur, jejichž stav se v čase mění a kde jsou jednotlivé operace vzájemně závislé.

Z hlediska návrhu algoritmů a datových struktur je amortizovaná analýza přínosná i tím, že umožňuje lépe vysvětlit, proč mohou být některá řešení v praxi efektivní, i když obsahují občasné nákladné kroky. Pokud je totiž celková cena dlouhé posloupnosti operací příznivá, může být daná datová struktura nebo algoritmus vhodným řešením i přesto, že jednotlivé operace nemají vždy malou cenu nejhoršího případu.

2.3.3 Srovnání s ostatními přístupy

Amortizovanou analýzu je vhodné odlišit od dalších běžných přístupů používaných při hodnocení algoritmů. Nejhorší případ sleduje maximální cenu jedné operace nebo jednoho běhu algoritmu. Průměrný případ naopak vyjadřuje očekávanou cenu vzhledem k určitému rozdělení vstupů. Amortizovaná analýza se od obou těchto přístupů liší tím, že hodnotí celou posloupnost operací a z ní odvozuje náklady připadající na jednu operaci [2, 4].

Je důležité zdůraznit, že amortizovaná složitost není totéž co složitost průměrná. Průměrná složitost vychází z pravděpodobnostního modelu vstupů, zatímco amortizovaná analýza žádné takové předpoklady nepotřebuje. Nejedná se tedy o statistický průměr přes různé vstupy, ale o rozložení nákladů v rámci posloupnosti operací. Právě v tom spočívá její významná odlišnost i její praktická síla [2].

Pro lepší přehlednost shrnuje tabulka 2.3.1 základní rozdíly mezi třemi běžnými přístupy k hodnocení náročnosti algoritmů.

Tabulka 2.3.1: Srovnání vybraných přístupů k hodnocení složitosti

Přístup	**Co sleduje**	**Charakteristika**
Nejhorší případ	Maximální cenu jedné operace nebo jednoho běhu	Poskytuje bezpečnou horní mez, ale může být příliš pesimistický.
Průměrný případ	Průměrnou cenu vzhledem k rozdělení vstupů	Závisí na předpokladech o vstupních datech a jejich pravděpodobnosti.
Amortizovaná analýza	Celkové náklady posloupnosti operací	Rozkládá cenu dražších operací mezi větší počet levnějších operací a nevyžaduje pravděpodobnostní model vstupů.

Ze srovnání vyplývá, že amortizovaná analýza zaujímá mezi ostatními přístupy zvláštní postavení. Zachovává výhodu pohledu nejhoršího případu v tom, že neposkytuje pouze optimistický odhad založený na pravděpodobnosti, ale zároveň dovoluje přesněji zachytit skutečné dlouhodobé chování algoritmu. Díky tomu je vhodná pro analýzu datových struktur, u nichž se stav postupně mění a kde některé operace ovlivňují cenu operací následujících [4].

Na základní vymezení pojmu amortizované složitosti navazují další části této kapitoly, které se podrobněji věnují jednotlivým metodám amortizované analýzy a jejich praktickému použití.

2.4 Metody amortizované analýzy

V této podkapitole jsou stručně popsány základní metody amortizované analýzy, které se využívají při hodnocení algoritmů a datových struktur. Pozornost je věnována agregované metodě, účetní metodě a potenciálové metodě. Na závěr jsou shrnuty jejich základní vlastnosti a vzájemné rozdíly.

2.4.1 Agregovaná metoda

Agregovaná metoda představuje nejjednodušší způsob amortizované analýzy. Jejím základním principem je sledování celkových nákladů celé posloupnosti operací a následné odvození průměrných nákladů připadajících na jednu operaci. Místo samostatného hodnocení jednotlivých operací se tedy analyzuje celkový součet jejich cen. Pokud lze ukázat, že posloupnost m operací má celkovou cenu $O(m)$, je možné usoudit, že amortizovaná cena jedné operace je konstantní [2, 5].

Výhodou agregované metody je její jednoduchost a přehlednost. Hodí se zejména v situacích, kdy lze celkové náklady celé posloupnosti vyjádřit přímo a bez zavádění dalších pomocných veličin. Naopak u složitějších datových struktur bývá tato metoda méně flexibilní [5].

2.4.2 Účetní metoda

Účetní metoda, označovaná také jako bankéřská metoda, vychází z představy, že jednotlivým operacím lze přiřadit určitou účtovanou cenu, která se může lišit od jejich skutečné ceny. Levnějším operacím je přiřazena vyšší cena, než jakou ve skutečnosti mají, a vzniklý přebytek se uchovává jako rezerva pro budoucí dražší operace. Díky tomu je možné rozložit cenu nákladnějších kroků mezi větší počet kroků, které jsou levnější [2, 5].

U této metody je důležité, aby nahromaděná rezerva v průběhu celé posloupnosti operací nikdy neklesla pod nulu. Účetní metoda tak názorně ukazuje, že některé operace si mohou vytvářet rezervu pro budoucí nákladnější kroky [5].

2.4.3 Potenciálová metoda

Potenciálová metoda pracuje s potenciálovou funkcí, která vyjadřuje stav datové struktury a určitou rezervu nahromaděnou v systému. Amortizovaná cena operace je zde určena jako součet její skutečné ceny a změny potenciálu mezi stavem před operací a po ní [2, 5]. Vzhledem k její obecnosti a významu bude tato metoda podrobněji vysvětlena v následující části kapitoly.

2.4.4 Vztah mezi jednotlivými metodami

Všechny tři metody sledují stejný cíl, tedy získání vhodného odhadu nákladů dlouhé posloupnosti operací. Liší se však způsobem, jakým k tomuto cíli docházejí. Agregovaná metoda pracuje s celkovým součtem nákladů celé posloupnosti, účetní metoda zavádí účtované ceny a rezervy a potenciálová metoda využívá potenciálovou funkci popisující stav datové struktury [2, 5, 4].

Pro lepší přehlednost shrnuje tabulka 2.4.1 základní vlastnosti těchto metod.

Tabulka 2.4.1: Základní metody amortizované analýzy

Metoda	**Základní princip**	**Charakteristika**
Agregovaná metoda	Sleduje celkovou cenu celé posloupnosti operací	Nejjednodušší přístup, vhodný pro případy, kdy lze celkové náklady snadno vyjádřit.
Účetní metoda	Přiřazuje operacím účtované ceny a vytváří rezervu pro dražší operace	Umožňuje rozložit cenu nákladných operací mezi levnější operace v posloupnosti.
Potenciálová metoda	Pracuje s potenciálovou funkcí vyjadřující stav datové struktury	Obecný a formální přístup, vhodný pro přesnější analýzu složitějších struktur.

Z uvedeného srovnání vyplývá, že jednotlivé metody nejsou ve vzájemném rozporu, ale spíše představují různé pohledy na tentýž problém. Zatímco agregovaná metoda bývá nejjednodušší na použití, potenciálová metoda nabízí největší obecnost a formální sílu. Právě proto je v další části této kapitoly podrobněji rozebrána.

2.5 Potenciálová metoda

V této části je podrobněji vysvětlena potenciálová metoda. Pozornost je věnována jejímu principu, způsobu vyjádření amortizované ceny operace a významu potenciálové funkce při hodnocení změn stavu datové struktury. V závěru této části jsou shrnuty hlavní výhody potenciálové metody ve srovnání s ostatními přístupy.

2.5.1 Princip potenciálové metody

Potenciálová metoda patří mezi základní metody amortizované analýzy a představuje obecný způsob, jak vyjádřit vztah mezi skutečnými náklady jednotlivých operací a jejich dlouhodobým chováním v celé posloupnosti operací. Vychází z předpokladu, že aktuální stav datové struktury může nést určitou rezervu pro budoucí nákladnější operace. Tato rezerva je popsána pomocí potenciálové funkce [4, 2, 5].

Potenciálová metoda proto nesleduje pouze skutečnou cenu právě prováděné operace, ale také to, jak se po jejím provedení změní stav celé datové struktury. Díky tomu umožňuje přesněji zachytit celkové chování delších posloupností operací než izolované hodnocení jednotlivých kroků. Ve srovnání s agregovanou a účetní metodou je abstraktnější, ale zároveň poskytuje obecný a formální nástroj, který je vhodný i pro složitější důkazy amortizované složitosti [5].

2.5.2 Potenciálová funkce a amortizovaná cena operace

Klíčovým pojmem této metody je potenciálová funkce, obvykle značená Φ. Tato funkce přiřazuje každému stavu datové struktury určitou číselnou hodnotu, která vyjadřuje velikost nahromaděné rezervy v daném stavu. Potenciál tedy nepředstavuje skutečný čas ani skutečný počet instrukcí, ale pomocnou veličinu, která umožňuje lépe vyjádřit rozložení nákladů mezi jednotlivé operace [4, 2].

Je-li skutečná cena i-té operace označena jako c_i, stav datové struktury před provedením operace jako D_{i-1} a stav po jejím provedení jako D_i, pak amortizovaná cena této operace může být zapsána ve tvaru

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Tento vztah vyjadřuje, že amortizovaná cena operace se skládá ze skutečné ceny operace a změny potenciálu mezi stavem před operací a po ní. Pokud se potenciál po operaci zvětší, znamená to, že se vytváří rezerva pro budoucí operace. Pokud se naopak potenciál zmenší, dochází k využití dříve nahromaděné rezervy [4, 5].

Důležitou vlastností potenciálové funkce je, že by měla být zvolena tak, aby její hodnoty byly nezáporné, případně aby počáteční stav neměl záporný potenciál. Díky tomu lze ukázat, že součet amortizovaných cen celé posloupnosti poskytuje horní odhad součtu jejich skutečných cen. Potenciálová metoda tak umožňuje dokázat, že i když některé operace mohou být drahé, jejich cena je v delší posloupnosti vyvážena operacemi jinými [2, 5].

Z praktického hlediska je při volbě potenciálové funkce důležité, aby odpovídala vlastnostem konkrétní datové struktury a správně vystihovala, jakým způsobem systém vytváří a spotřebovává rezervu. Právě volba vhodné potenciálové funkce bývá při důkazu amortizované složitosti jedním z nejdůležitějších kroků.

2.5.3 Význam a využití potenciálové metody

Význam potenciálové metody spočívá především v její obecnosti a formální síle. Ve srovnání s ostatními metodami amortizované analýzy umožňuje velmi přesně popsat změny stavu datové struktury a jejich vliv na cenu budoucích operací. Díky tomu je vhodná i pro analýzu složitějších struktur, u nichž nestačí pouze sledovat celkový součet nákladů nebo intuitivně rozdělovat rezervu mezi jednotlivé operace [4, 5].

Potenciálová metoda je užitečná zejména v situacích, kdy se stav datové struktury v čase výrazně mění a kdy některé operace přímo ovlivňují náročnost operací následujících. V takovém případě poskytuje přirozený způsob, jak vyjádřit, že nákladnější operace nejsou izolovaným jevem, ale souvisejí s vývojem celého systému. To je také důvod, proč je tato metoda často považována za nejuniverzálnější z běžně používaných metod amortizované analýzy.

Z hlediska této práce je potenciálová metoda důležitá i proto, že umožňuje názorně propojit teoretický princip amortizované analýzy s konkrétním chováním algoritmů a datových struktur.

Pomocí potenciálu lze sledovat, jak se v jednotlivých stavech vytváří nebo spotřebovává rezerva, a tím lépe vysvětlit, proč nelze cenu některých algoritmů hodnotit pouze podle jedné samostatné operace. Právě tato vlastnost činí potenciálovou metodu vhodnou nejen pro teoretickou analýzu, ale i pro výukové a vizualizační účely.

2.6 Vybrané algoritmy vhodné pro amortizovanou analýzu

V této podkapitole jsou stručně představeny algoritmy, u nichž je amortizovaná analýza vhodným nástrojem pro hodnocení jejich chování. Nejprve jsou uvedeny některé příklady algoritmů, u nichž se tento přístup běžně využívá. Následně jsou zmíněny algoritmy vybrané do praktické části této práce a na závěr jsou přehledně shrnuty.

2.6.1 Algoritmy vhodné pro amortizovanou analýzu

Amortizovaná analýza se uplatňuje zejména u algoritmů, u nichž se jednotlivé operace mohou lišit svou skutečnou cenou, avšak při delší posloupnosti operací vykazuje celý systém příznivější chování. Tento přístup je vhodný především tehdy, když se vyskytují občasné nákladnější operace, jejichž cenu lze rozložit mezi větší počet operací levnějších. Typickou situací je stav, kdy se mění datová struktura a některé operace ovlivňují náročnost operací následujících.

Mezi často uváděné příklady algoritmů vhodných pro amortizovanou analýzu patří například:

- Multipop na zásobníku,
- Fronta využívající dva zásobníky,
- Splay strom,
- Dynamické pole s občasným rozšířením kapacity,
- Binární čítač,
- některé operace nad disjoint-set strukturami.

Tyto algoritmy mají společné to, že jednotlivé operace nemusí mít vždy stejnou cenu, ale v delší posloupnosti operací lze jejich chování pomocí amortizované analýzy popsat přesněji, než kdyby bylo hodnoceno pouze izolovaně po jednotlivých krocích.

2.6.2 Algoritmy vybrané do praktické části práce

Do praktické části této bakalářské práce byly vybrány tři algoritmy: Multipop na zásobníku, fronta využívající dva zásobníky a Splay strom. Tyto algoritmy byly zvoleny tak, aby společně tvořily trojici příkladů různé obtížnosti a zároveň umožňovaly názorně ukázat různé projevy amortizované složitosti.

2.6.3 Přehled vybraných algoritmů

Pro lepší přehlednost shrnuje tabulka 2.6.1 algoritmy vybrané do praktické části práce.

Tabulka 2.6.1: Vybrané algoritmy vhodné pro amortizovanou analýzu

Algoritmus	**Obtížnost**	**Stručná charakteristika**
Multipop na zásobníku	jednoduchá	Jednoduchý příklad ukazující, že jedna dražší operace může být v delší posloupnosti nákladově vyvážena.
Fronta využívající dva zásobníky	střední	Příklad rozložení nákladů mezi operace enqueue a dequeue při implementaci fronty pomocí dvou zásobníků.
Splay strom	vyšší	Pokročilejší datová struktura, u níž se amortizovaná analýza používá pro popis chování po sérii přístupových operací.

Z uvedeného přehledu je patrné, že vybrané algoritmy pokrývají různé úrovně obtížnosti i různé způsoby projevu amortizované složitosti. Právě proto tvoří vhodný základ pro praktickou část této práce, v níž budou dále využity pro vytvoření názorné výukové vizualizace.

Kapitola 3

Návrh komponenty výukového serveru

Třetí kapitola se primárně věnuje návrhu komponenty výukového serveru zaměřené na problematiku amortizované složitosti. Nejprve popisuje cíl a roli navrhované komponenty v rámci výukového serveru a vysvětluje její přínos pro výuku této oblasti. Následně shrnuje hlavní požadavky, které byly na komponentu kladeny, a objasňuje výběr algoritmů zařazených do praktické části práce. Další část kapitoly se zaměřuje na návrh funkcionality komponenty, zejména na způsoby zadávání vstupů, zobrazení průběhu algoritmů a prezentaci hodnot důležitých pro pochopení amortizované analýzy. Na konci kapitoly je pozornost věnována také návrhu uživatelského rozhraní, celkové struktury komponenty a možného rozšíření.

3.1 Cíl a role navrhované komponenty

V této části je nejprve vymezeno zařazení navrhované komponenty v rámci výukového serveru. Následně je pozornost věnována hlavnímu účelu komponenty a její roli při zprostředkování problematiky amortizované složitosti studentům. V závěru této části je shrnut její přínos pro výuku a pro názornější pochopení sledované problematiky.

3.1.1 Zařazení komponenty do výukového serveru

Vytvořená komponenta vzniká jako součást výukového serveru určeného pro předměty TI. Tento výukový server má mít podobu sady dynamických webových stránek, jejichž hlavním cílem je umožnit budoucím studentům lépe pochopit principy různých typů úloh a problémů prostřednictvím názorných a interaktivních ukázek. Na rozdíl od běžných statických výukových materiálů, které obvykle pracují pouze s pevně daným počtem příkladů, má takové řešení umožnit generovat libovolný počet ukázkových příkladů vybraných algoritmů na základě vstupů od uživatele a sledovat jejich průběh v různých situacích.

Navrhovaná komponenta je v rámci tohoto serveru zaměřena na oblast amortizované složitosti. Jejím úkolem je rozšířit výukový server o část, která studentům umožní lépe pochopit principy tohoto typu analýzy na konkrétních algoritmech a datových strukturách. Komponenta se tedy nesnaží představit samostatný izolovaný nástroj, ale stává se součástí výuky v TI a zapadá do širšího konceptu interaktivní podpory studia teoretické informatiky.

3.1.2 Hlavní účel komponenty

Hlavním účelem této navržené komponenty je vytvořit prostředek pro pochopení počítání složitosti vybraných algoritmů s využitím amortizované analýzy, která byla podrobně popsána v předchozí kapitole. Komponenta má budoucím studentům poskytnout vizualizaci toho, jak se algoritmus chová při zpracování vstupu, jak se mění stav dané datové struktury a jak probíhá vývoj hodnot v jednotlivých krocích, které jsou důležité pro amortizovanou analýzu. Důraz je kladen především na to, aby uživatel viděl podrobný průběh algoritmu v každém kroku, nikoli pouze konečný výsledek výpočtu.

Účelem komponenty tedy není pouze umožnit spuštění algoritmu, ale především podpořit hlubší pochopení celého principu, proč se amortizovaná složitost hodnotí v rámci delší posloupnosti operací. Komponenta má propojit teoretický výklad s názornou ukázkou chování algoritmu a nabídnout studentovi možnost sledovat souvislosti mezi jednotlivými operacemi a jejich dlouhodobým dopadem.

3.1.3 Přínos komponenty pro výuku amortizované složitosti

Celá problematika amortizované složitosti bývá pro studenty často obtížně pochopitelná, protože její správné pochopení vyžaduje sledování souvislostí mezi více operacemi, změn stavů vizualizovaných datových struktur a vývoje hodnot, které se při amortizované analýze používají. Klasické výklady v teoretické formě pomocí prezentací, skript nebo několika málo statických ukázek nemusí vždy stačit k tomu, aby si studenti vytvořili dostatečně jasnou představu o tom, proč se některé operace mohou jevit jako nákladné oproti ostatním a proč je přesto možné celkové chování algoritmu hodnotit příznivěji.

Důležitý přínos této komponenty spočívá především v propojení teoretického výkladu s názornou praktickou ukázkou. Student může při práci s komponentou aktivně sledovat průběh algoritmů, pozorovat změny stavů datových struktur a lépe chápat vztah mezi jednotlivými operacemi a jejich dlouhodobým chováním. Komponenta tak může přispět k názornější a srozumitelnější výuce amortizované složitosti v rámci předmětů TI.

3.2 Požadavky na komponentu

Tato část se zaměřuje na hlavní požadavky, které byly na navrhovanou komponentu kladeny. Nejprve jsou shrnuty funkční požadavky související s chováním a možnostmi komponenty. Následně jsou popsány požadavky na uživatelskou interakci a na způsob, jakým má uživatel s komponentou pracovat. Závěr této části je věnován požadavkům na názornost a výukovou hodnotu navrhovaného řešení.

3.2.1 Funkční požadavky

Při návrhu celé komponenty bylo potřeba vycházet z požadavků, které byly stanoveny v zadání bakalářské práce. Základním funkčním požadavkem bylo vytvořit dynamické webové stránky, které umožní vhodným způsobem zobrazit simulaci běhu alespoň tří různých algoritmů, u nichž je vhodné analyzovat složitost pomocí amortizované analýzy. Komponenta tak měla uživateli umožnit nejen samotné spuštění algoritmu, ale také sledování jeho jednotlivých kroků v průběhu zpracování zadaného vstupu.

Dalším důležitým požadavkem bylo umožnit uživateli zadávat vstupy více různými způsoby. Návrh u každého algoritmu proto musel počítat s možností ručního zadávání vstupů, s generováním náhodných vstupů podle zvolených parametrů, s prací s různě velkými ukázkovými vstupy a také se zadáváním vstupů pomocí syntaxe příkazů. Komponenta měla zároveň během simulace zobrazovat informace o aktuálním stavu výpočtu, počet provedených kroků a aktuální hodnotu potenciálu nebo naspořených mincí podle použité metody amortizované analýzy. V této práci byla zvolena potenciálová metoda.

Pro lepší přehlednost shrnuje tabulka 3.2.1 hlavní funkční požadavky navrhované komponenty.

Tabulka 3.2.1: Hlavní funkční požadavky navrhované komponenty

Požadavek	**Stručný popis**
Podpora více algoritmů	Komponenta musí umožnit práci alespoň se třemi algoritmy vhodnými pro amortizovanou analýzu.
Různé způsoby zadávání vstupů	Uživatel musí mít možnost zadávat vstupy ručně, náhodně je generovat, pracovat s ukázkovými vstupy různé velikosti a zadávat je také pomocí syntaxe.
Zobrazení průběhu algoritmu	Komponenta musí umožnit sledovat chování algoritmu v průběhu výpočtu, nikoli pouze jeho konečný výsledek.
Zobrazení stavu datové struktury	V průběhu simulace musí být zobrazen aktuální stav datové struktury používané algoritmem.
Zobrazení doplňujících hodnot	Součástí simulace musí být zobrazení počtu kroků a hodnot souvisejících s amortizovanou analýzou, například potenciálu nebo naspořených mincí.
Podpora teoretického výkladu	Ke každému algoritmu musí být dostupné také odvození jeho složitosti jako doplnění interaktivní simulace.

Dalším funkčním požadavkem bylo také to, aby ke každému algoritmu bylo přímo v rámci vytvořené stránky, například ve formě statické části, dostupné odvození jeho složitosti. Celá komponenta podle zadání neměla sloužit pouze jako prostředek pro interaktivní simulaci, ale také jako podpora teoretického výkladu jednotlivých algoritmů a jejich amortizované analýzy.

3.2.2 Požadavky na uživatelskou interakci

Funkčnost nebyla jedinou důležitou prioritou, ale bylo také nutné navrhnout komponentu tak, aby s ní dokázal pracovat každý uživatel přehledně a srozumitelně. Celý vzhled stránky měl působit přívětivě a zadávání vstupů mělo být navrženo tak, aby bylo přehledné a srozumitelné. Uživatel by neměl mít problém zadat libovolný vstup a v případě chyby při zadávání by měl být vhodným způsobem upozorněn na to, co zadává nesprávně.

Návrh zároveň počítal s tím, že uživatel bude moci měnit vstupy, opakovaně spouštět simulace a sledovat, jak se změny vstupu projeví v průběhu algoritmu. Důležitým požadavkem bylo také to, aby uživatel mohl snadno sledovat jednotlivé kroky výpočtu a rozuměl tomu, co se v daném okamžiku děje. Interakce s komponentou proto neměla být omezena pouze na zobrazení výsledku, ale měla podporovat průběžné vnímání změn stavu datové struktury a souvisejících hodnot.

3.2.3 Požadavky na názornost a výukovou hodnotu

Jedním z nejdůležitějších požadavků na navrhovanou komponentu byla její názornost a výuková hodnota. Nestačilo, aby komponenta technicky zobrazovala průběh algoritmu, ale bylo také nutné, aby uživateli pomáhala pochopit celou problematiku. Návrh proto musel směřovat k tomu, aby student mohl jasně sledovat vztah mezi jednotlivými operacemi, změnami stavu datové struktury a vývojem hodnot důležitých pro použitou metodu analýzy.

Výuková hodnota komponenty spočívá především v propojení teoretického výkladu s praktickou ukázkou. Uživatel nemá pouze číst popis algoritmu, ale má současně vidět jeho konkrétní chování při různých typech vstupů a lépe chápat, proč se celkové náklady algoritmu neposuzují jen podle jedné operace. Proto bylo při návrhu důležité dbát na srozumitelnost, přehlednost a na takové uspořádání informací, které podpoří lepší orientaci uživatele v průběhu simulace.

3.3 Výběr algoritmů pro praktickou část

V této části je vysvětlen výběr algoritmů zařazených do praktické části práce. Nejprve jsou uvedena hlavní kritéria, podle nichž byly algoritmy vybírány. Následně jsou představeny jednotlivé zvolené algoritmy a je objasněno, proč byly vybrány právě jako zástupci různé obtížnosti a různých projevů amortizované složitosti.

3.3.1 Kritéria výběru algoritmů

Výběr algoritmů byl velmi důležitý, protože bylo potřeba zvolit takové příklady, které budou vhodné z hlediska amortizované analýzy a současně budou přínosné pro výuku. Jedním z hlavních kritérií bylo to, že u zvolených algoritmů nebo datových struktur se mohou jednotlivé operace výrazně lišit svou skutečnou cenou, avšak při delší posloupnosti operací lze jejich chování hodnotit příznivěji pomocí amortizované analýzy.

Dalším důležitým kritériem byla rozdílná obtížnost vybraných algoritmů. Cílem bylo zařadit takové příklady, které studentům umožní postupovat od jednoduššího principu k náročnějšímu. Z tohoto důvodu byl vybrán jeden jednodušší algoritmus, jeden středně obtížný příklad a jedna pokročilejší datová struktura. Výběr byl zároveň přizpůsoben tomu, aby bylo možné jednotlivé algoritmy vhodně vizualizovat a názorně na nich ukázat různé projevy amortizované složitosti.

3.3.2 Multipop na zásobníku jako jednoduchý příklad

Jako jednodušší příklad byl zvolen algoritmus Multipop na zásobníku. Tento příklad je pro výuku vhodný především proto, že na něm lze poměrně snadno ukázat základní princip amortizované analýzy. Zásobník je navíc datová struktura, která je studentům obvykle dobře známá, a proto není nutné věnovat velkou pozornost jejím základním vlastnostem. Díky tomu je možné soustředit se především na samotný princip analýzy nákladů jednotlivých operací.

V rámci tohoto příkladu se nad zásobníkem provádějí zejména operace *Push*, *Pop* a *Multipop*. Operace *Push* vloží prvek na vrchol zásobníku a operace *Pop* jeden prvek z vrcholu odebere. Obě tyto operace mají konstantní časovou složitost $O(1)$. Operace *Multipop* však může v jednom kroku odebrat více prvků najednou, a proto může mít v nejhorším případě lineární složitost vzhledem k počtu odebíraných prvků. Přesto lze ukázat, že v delší posloupnosti operací je amortizovaná cena jednotlivých operací konstantní, protože každý prvek může být ze zásobníku odstraněn pouze omezený početkrát. Právě tato vlastnost činí z Multipopu na zásobníku vhodný úvodní příklad pro pochopení amortizované analýzy [6].

3.3.3 Fronta využívající dva zásobníky jako středně obtížný příklad

Jako středně obtížný příklad byla zvolena fronta implementovaná pomocí dvou zásobníků. Tento příklad je vhodný především proto, že kombinuje známé principy datových struktur LIFO a FIFO a současně ukazuje, že i operace, která může být v konkrétním okamžiku nákladná, může mít při delší posloupnosti operací příznivou amortizovanou složitost.

Při této implementaci se pro realizaci fronty používají dva zásobníky. Do prvního zásobníku se prvky vkládají operací *Enqueue*, zatímco odebírání pomocí operace *Dequeue* probíhá ze zásobníku druhého. Pokud je druhý zásobník prázdný, musí se prvky z prvního zásobníku do druhého postupně přesunout. Operace *Enqueue* má konstantní složitost $O(1)$, zatímco operace *Dequeue* může mít v nejhorším případě lineární složitost $O(n)$, pokud je nutné převést větší počet prvků. Z dlouhodobého hlediska však každý prvek projde tímto přesunem pouze jednou, a proto lze ukázat, že obě základní operace mají konstantní amortizovanou cenu. Tento algoritmus tak představuje vhodný střední stupeň mezi jednoduchým příkladem Multipopu a pokročilejší strukturou v podobě Splay stromu [7].

3.3.4 Splay strom jako pokročilejší příklad

Jako pokročilejší příklad byl zvolen Splay strom, tedy samočinně upravující se binární vyhledávací strom BST. Tento příklad představuje náročnější datovou strukturu, protože zde amortizovaná analýza neslouží pouze k vysvětlení jedné pomocné operace, ale k popisu dlouhodobého chování celé struktury při sérii přístupových operací.

Základní myšlenkou Splay stromu je, že po každém přístupu k uzlu dochází k jeho přesunu směrem ke kořeni pomocí rotací. Využívají se přitom operace typu *Zig*, *Zig-Zig* a *Zig-Zag*. Mezi základní operace nad touto strukturou patří *Search*, *Insert* a *Delete*. Operace *Search* slouží k vyhledání prvku ve stromu, operace *Insert* k vložení nového prvku a operace *Delete* k jeho odstranění. Pro Splay strom je typické, že po provedení těchto operací dochází k přesunu navštíveného nebo upravovaného uzlu směrem ke kořeni, čímž se mění struktura stromu s ohledem na budoucí přístupy.

Jedna konkrétní operace může být v nejhorším případě časově náročná a může dosáhnout až lineární složitosti $O(n)$, pokud se hledaný nebo upravovaný uzel nachází hluboko ve stromu. Přesto lze ukázat, že při delší posloupnosti operací mají základní operace nad Splay stromem amortizovanou složitost $O(\log n)$. Splay strom je proto vhodným příkladem pokročilejšího využití amortizované analýzy v oblasti dynamických datových struktur a tvoří přirozený vrchol trojice vybraných algoritmů [8].

3.4 Návrh funkcionality komponenty

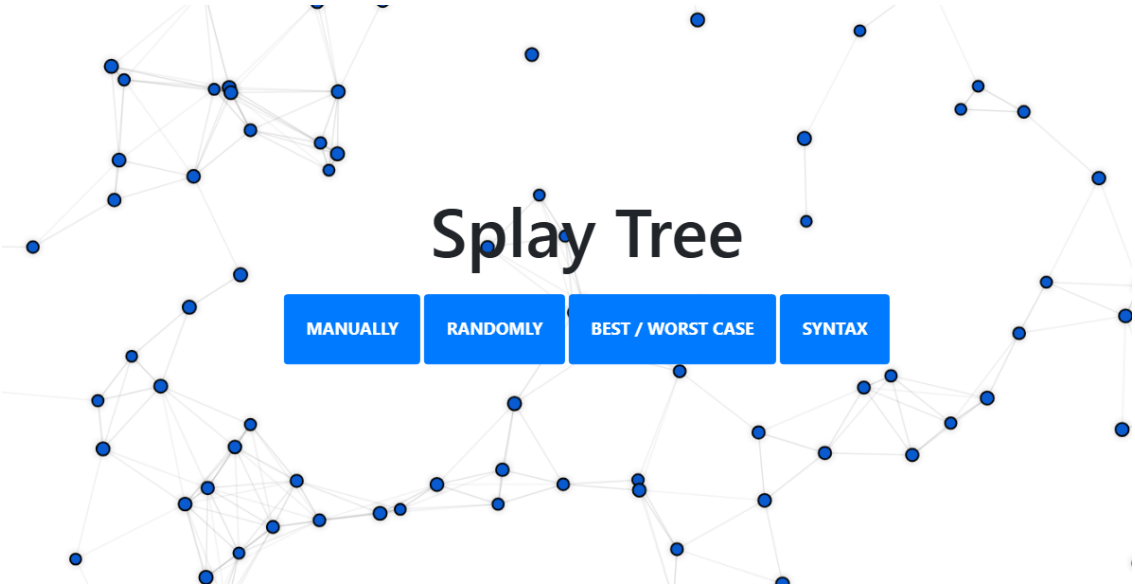
Tato část se věnuje návrhu hlavních funkcí, které má komponenta uživateli poskytovat. Pozornost je zaměřena především na způsoby zadávání vstupů, na zobrazení průběhu algoritmu po jednotlivých krocích a na prezentaci hodnot důležitých pro pochopení amortizované analýzy. Součástí této části je také návrh doplňujících informací o algoritmech, které mají uživateli usnadnit orientaci.

3.4.1 Režimy zadávání vstupů

Velmi důležitou součástí návrhu funkcionality komponenty bylo umožnit uživateli pracovat s více způsoby zadávání vstupů. Pro vyzkoušení vybraných algoritmů by byl pouze jeden způsob zadávání vstupů nedostačující, a proto bylo vhodné navrhnout více typů vstupu, které budou odpovídat různým potřebám uživatele i různým výukovým situacím. Komponenta proto byla navržena tak, aby umožňovala ruční zadávání vstupů, generování náhodných vstupů podle zvolených parametrů, práci s ukázkovými vstupy reprezentujícími typické situace a také zadávání vstupů pomocí syntaxe příkazů.

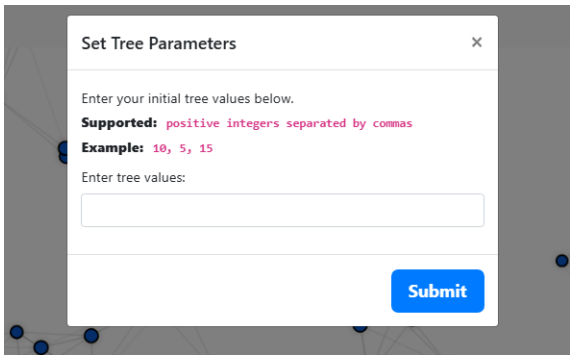
Tyto navržené typy zadávání vstupů umožňují studentovi pracovat s komponentou aktivnějším způsobem. Uživatel si může sám zvolit, zda chce sledovat předem připravený příklad, experimentovat s náhodně vytvořeným vstupem, nebo si přesně nadefinovat vlastní scénář pomocí ručně zadaných hodnot nebo posloupnosti příkazů. Návrh této části funkcionality tak podporuje jak samostatné

objevování chování algoritmů, tak cílené sledování konkrétních situací důležitých pro pochopení amortizované složitosti.

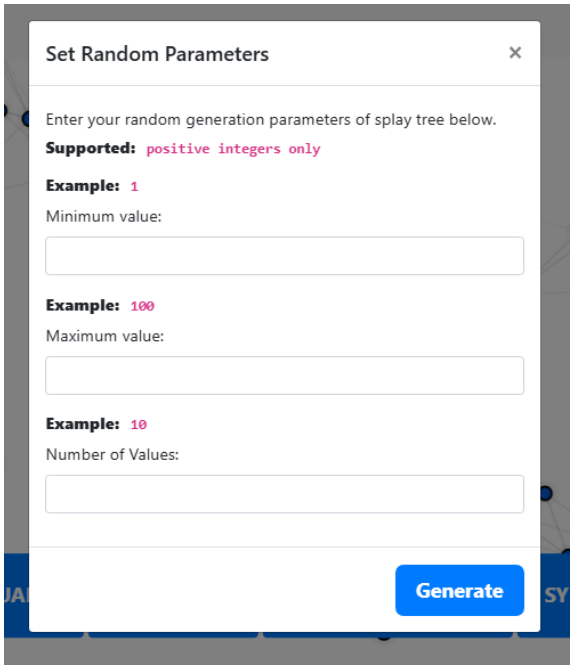


Obrázek 3.4.1: Hlavní nabídka režimů zadávání vstupů u zvoleného algoritmu

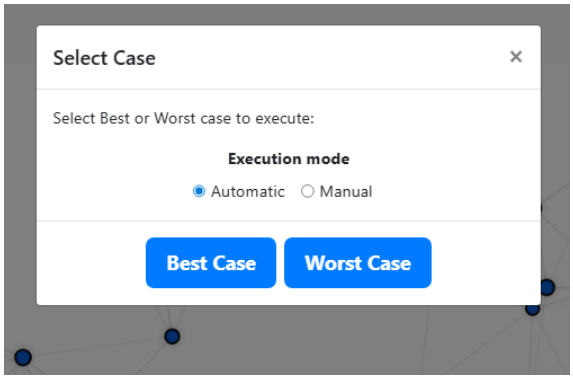
Na obrázku 3.4.1 je znázorněna hlavní nabídka režimů zadávání vstupů, která je uživateli k dispozici po zvolení konkrétního algoritmu. Obrázek 3.4.2 pak ukazuje konkrétní podobu jednotlivých režimů zadávání, konkrétně ruční zadávání vstupů, náhodné generování, výběr nejlepšího nebo nejhoršího případu a zadávání vstupů pomocí syntaxe příkazů.



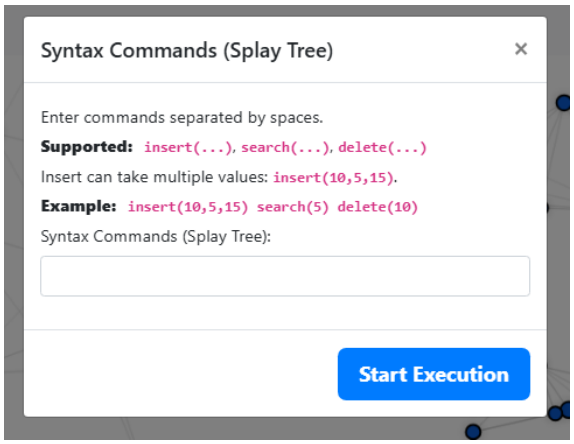
(a) Ruční zadávání vstupů



(b) Náhodné generování vstupů



(c) Volba nejlepšího/nejhoršího případu



(d) Zadávání vstupů pomocí syntaxe příkazů

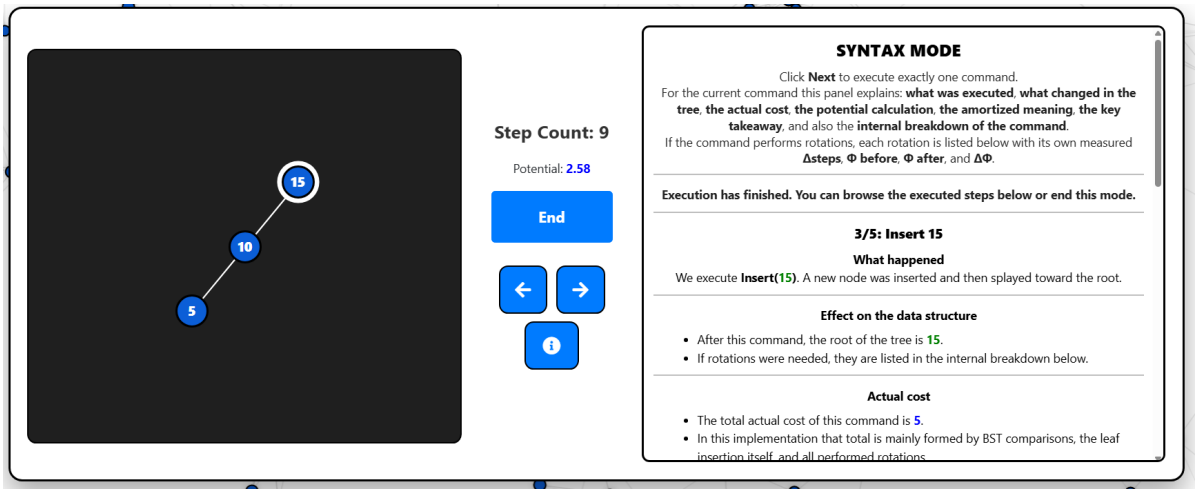
Obrázek 3.4.2: Příklady jednotlivých režimů zadávání vstupů

3.4.2 Zobrazení průběhu algoritmu po jednotlivých krocích

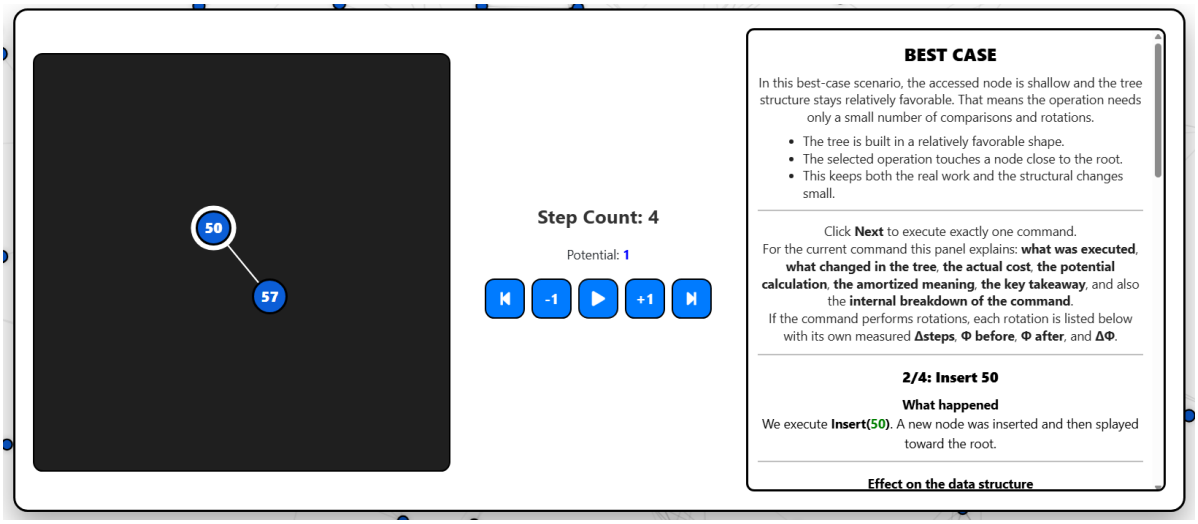
Další důležitou součástí návrhu funkcionality komponenty bylo zobrazení průběhu algoritmu po jednotlivých krocích. Pro výuku amortizované složitosti totiž nestačí uživateli ukázat pouze konečný výsledek, ale je nutné umožnit mu sledovat, co se děje v průběhu celého výpočtu. Komponenta proto byla navržena tak, aby uživatel mohl vidět postupný vývoj algoritmu a změny stavu datové struktury v jednotlivých krocích simulace.

Takové zobrazení je důležité především proto, že studentovi umožňuje lépe pochopit návaznost jednotlivých operací a jejich vliv na další chování algoritmu. Díky krokovému průběhu lze sledovat nejen samotný výsledek operace, ale i způsob, jakým k němu algoritmus dospívá. Návrh této funkcionality proto představuje jednu z klíčových částí celé komponenty.

Na obrázku 3.4.3 je znázorněn průběh algoritmu v režimu syntaxe, kde uživatel postupuje jednotlivými kroky simulace a může sledovat aktuální stav datové struktury i informace o právě prováděné operaci. Obrázek 3.4.4 pak ukazuje automatický průběh v režimu nejlepšího/nejhoršího případu, kde se předem připravený scénář vykonává postupně bez nutnosti ručního zadávání každého kroku.



Obrázek 3.4.3: Zobrazení průběhu algoritmu po jednotlivých krocích v režimu syntaxe



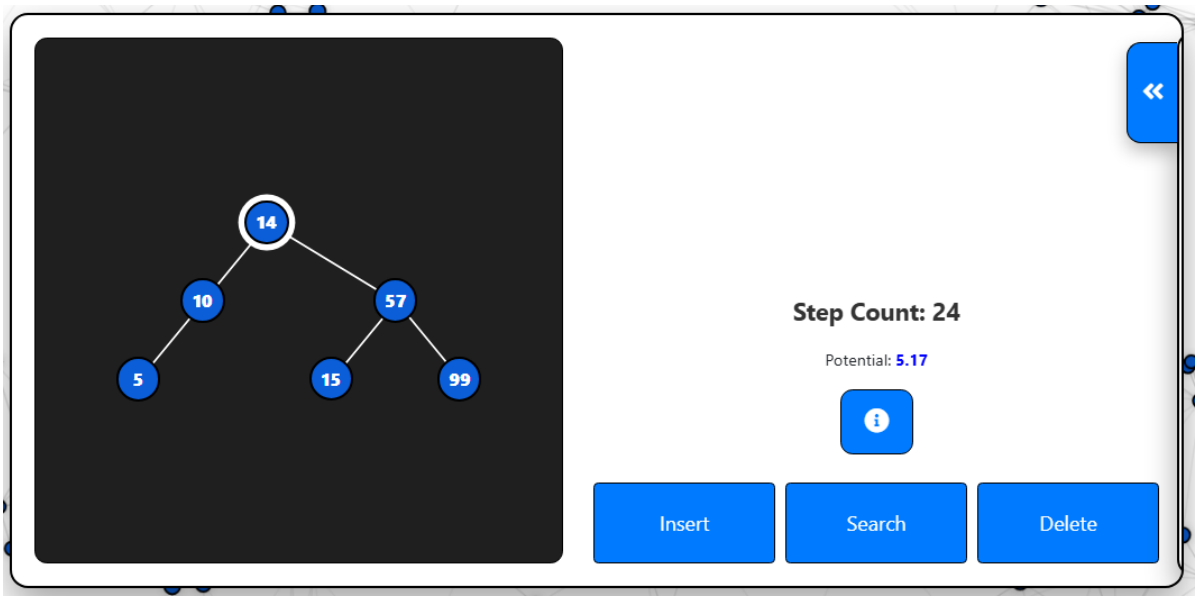
Obrázek 3.4.4: Automatický průběh simulace v režimu nejlepšího/nejhoršího případu

3.4.3 Zobrazení počtu kroků a potenciálu

Pro správné pochopení amortizované analýzy bylo při návrhu důležité zohlednit také zobrazení hodnot, které s tímto typem hodnocení přímo souvisejí. Komponenta proto měla zobrazovat počet provedených kroků a zároveň i hodnotu potenciálu, případně jinou odpovídající veličinu podle použité metody analýzy. Tyto hodnoty nemají sloužit pouze jako doplněk vizualizace, ale jako důležitá součást výkladu toho, proč se chování algoritmu hodnotí určitým způsobem.

Zobrazení počtu kroků a potenciálu umožňuje uživateli sledovat nejen to, co algoritmus právě vykonává, ale také to, jak se mění hodnoty důležité pro celkovou interpretaci jeho amortizované složitosti. Student tak může lépe pochopit vztah mezi jednotlivými operacemi, jejich skutečnými náklady a dlouhodobým chováním algoritmu v rámci celé posloupnosti operací.

Na obrázku 3.4.5 je ukázáno zobrazení počtu provedených kroků a aktuální hodnoty potenciálu v průběhu simulace. Tyto informace jsou součástí hlavního rozhraní komponenty a tvoří důležitou oporu při sledování vývoje algoritmu i při interpretaci jeho amortizované složitosti.



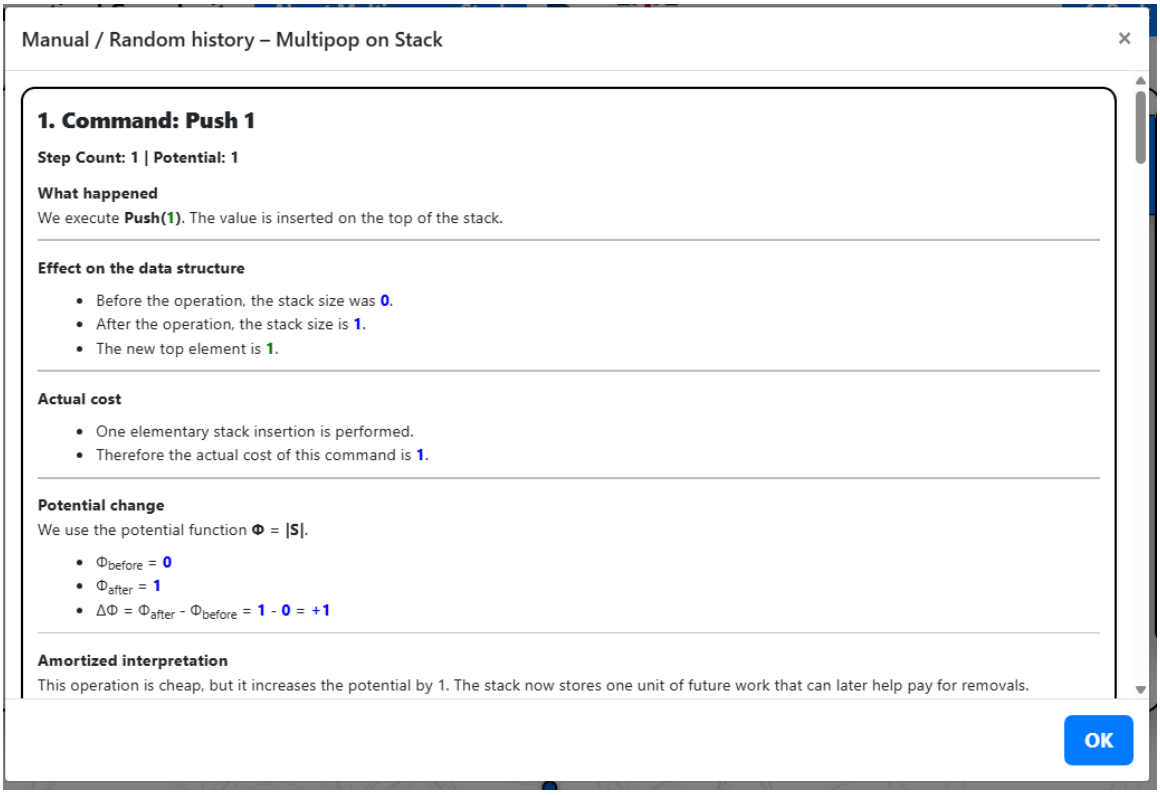
Obrázek 3.4.5: Zobrazení počtu kroků a aktuální hodnoty potenciálu v průběhu simulace

3.4.4 Zobrazení doplňujících informací o aktuálním stavu algoritmu

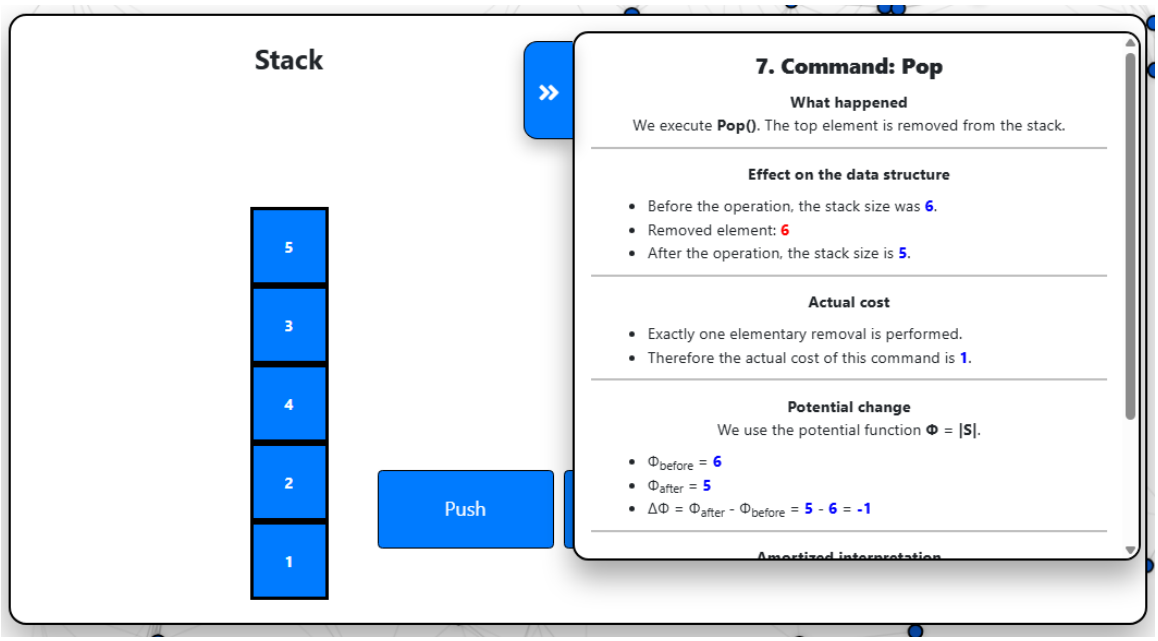
Součástí návrhu funkcionality komponenty bylo také zobrazení doplňujících informací o aktuálním stavu algoritmu. Samotná vizualizace datové struktury nebo zobrazení provedené operace totiž nemusí být vždy dostačující k tomu, aby uživatel přesně porozuměl tomu, co se v daném okamžiku děje. Z tohoto důvodu bylo vhodné doplnit průběh simulace o textové a přehledně uspořádané informace, které uživateli usnadní orientaci.

Tyto doplňující informace mají pomoci objasnit význam aktuálního kroku, stav datové struktury nebo souvislosti mezi právě prováděnou operací a chováním algoritmu jako celku. Uživatel tak nemá k dispozici pouze vizuální podobu aktuálního stavu, ale také slovní vysvětlení toho, co bylo provedeno, co se změnilo a jaký dopad má daný krok z hlediska amortizované analýzy. Návrh této části funkcionality tak podporuje srozumitelnost komponenty a přispívá k tomu, aby uživatel mohl lépe propojit vizuální průběh simulace s teoretickým výkladem algoritmu.

Na obrázku 3.4.6 je znázorněn příklad detailních informací dostupných v historii vykonaných kroků. Obrázek 3.4.7 pak ukazuje doplňující informační panel přímo v průběhu práce s algoritmem, kde jsou zobrazeny informace vztahující se k aktuálně prováděné operaci.



Obrázek 3.4.6: Detailní doplňující informace dostupné v historii provedených kroků



Obrázek 3.4.7: Doplňující informační panel zobrazující aktuální stav operace při práci s algoritmem

3.5 Návrh uživatelského rozhraní

V této části je popsán návrh UI navrhované komponenty. Nejprve je pozornost věnována návrhu hlavní stránky a způsobu výběru jednotlivých algoritmů. Následně jsou shrnuty základní ovládací prvky simulace a způsob vizualizace použitých datových struktur. V závěru této části je popsán návrh informačních panelů a doprovodných vysvětlení, která mají podpořit výukový charakter komponenty.

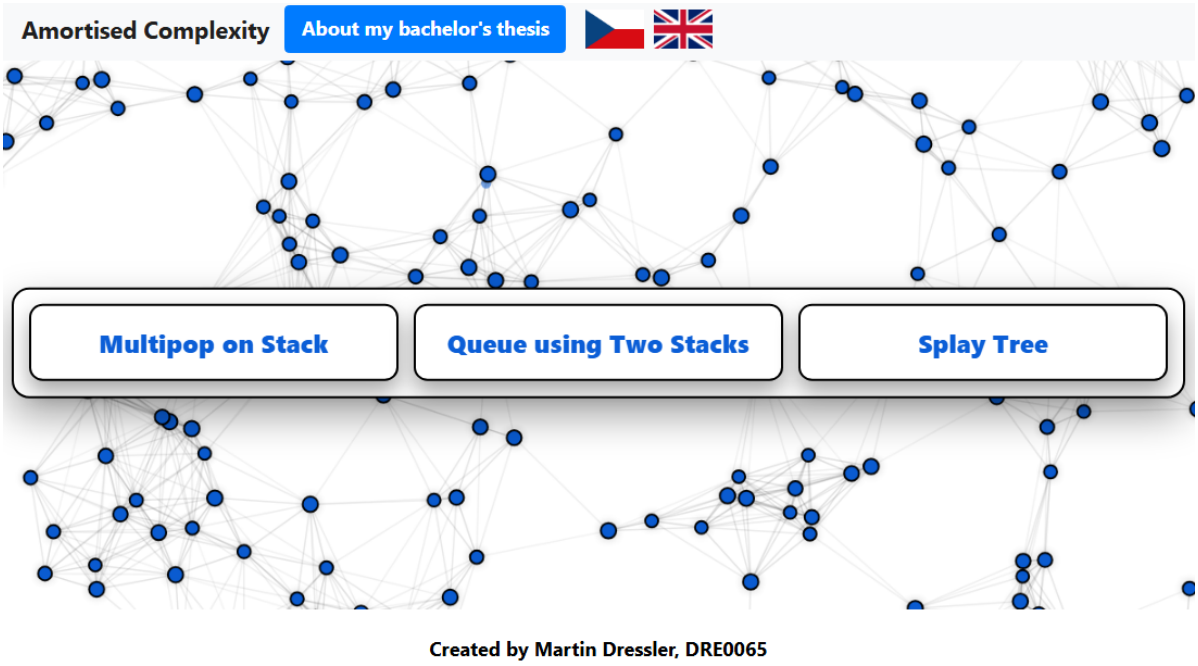
3.5.1 Návrh hlavní stránky a výběru algoritmu

Při návrhu uživatelského rozhraní bylo důležité vytvořit hlavní stránku, která bude pro uživatele přehledná a umožní mu rychlou orientaci v celé komponentě. Hlavní stránka proto byla navržena jako výchozí rozcestník, z něhož si uživatel vybírá konkrétní algoritmus, se kterým chce dále pracovat. Současně bylo cílem, aby již na této úrovni bylo zřejmé, že komponenta slouží jako součást výukového prostředí zaměřeného na amortizovanou složitost.

Velký důraz byl kladen na jednoduchost výběru jednotlivých algoritmů. Uživatel má mít možnost z hlavní stránky přímo přejít k vybranému algoritmu, aniž by musel procházet složitější strukturu nabídek. Návrh proto počítal s výrazným zobrazením všech tří algoritmů zařazených do praktické části práce. Tím je zajištěno, že uživatel ihned vidí, jaké algoritmy jsou v komponentě dostupné, a může si zvolit ten, který chce právě vyzkoušet.

Součástí návrhu hlavní stránky byly také další prvky podporující orientaci a dostupnost informací. Patří mezi ně zejména možnost přepínání jazykových preferencí a přístup k doplňujícím informacím o samotné bakalářské práci. Hlavní stránka tak neslouží pouze jako technický vstup do aplikace, ale zároveň plní orientační a informační funkci.

Na obrázku 3.5.1 je znázorněna hlavní stránka komponenty, na níž jsou zobrazeny dostupné algoritmy, přepínač jazykových preferencí a tlačítko pro zobrazení informací o bakalářské práci.



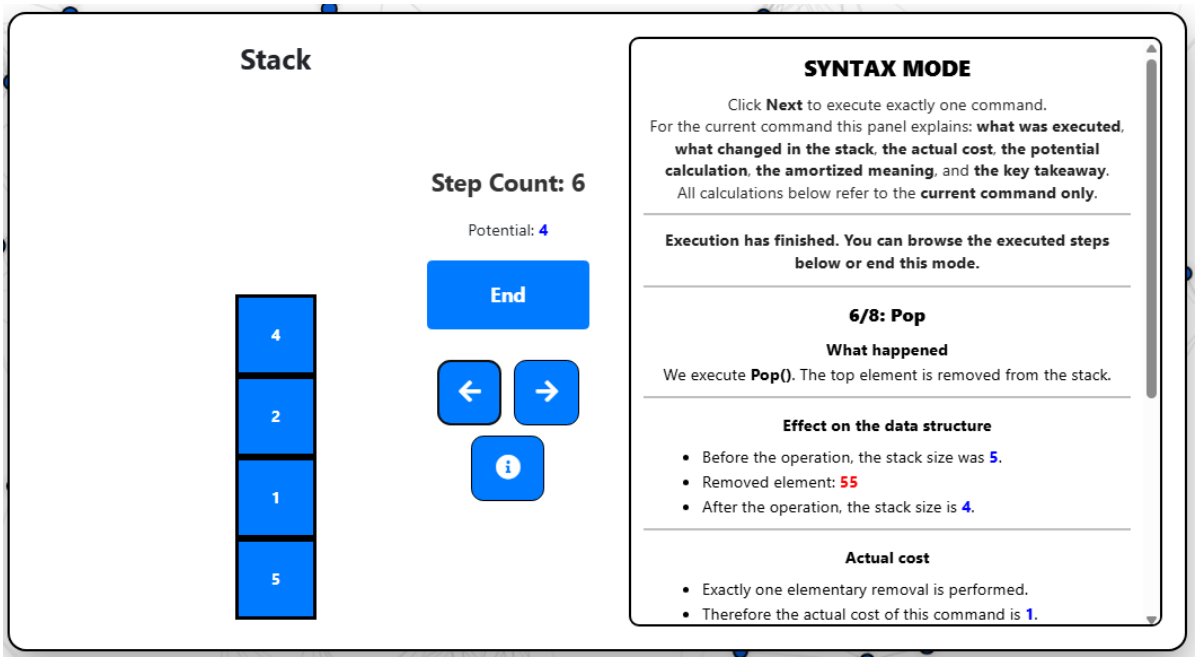
Obrázek 3.5.1: Hlavní stránka komponenty s výběrem algoritmů, přepínačem jazykových mutací a tlačítkem pro informace o bakalářské práci

3.5.2 Návrh ovládacích prvků simulace

Součástí návrhu uživatelského rozhraní bylo také vytvoření vhodných ovládacích prvků simulace. Tyto prvky musely uživateli umožnit pohodlně řídit průběh algoritmu a současně podporovat srozumitelné sledování jednotlivých kroků. Návrh proto počítal s tím, že uživatel nebude pouze pasivně sledovat vykonávání algoritmu, ale bude moci průběh simulace aktivně ovlivňovat.

Důležitou součástí ovládání bylo zejména krokování mezi jednotlivými stavy simulace. Uživatel má mít možnost přecházet na předchozí nebo následující krok, ukončit aktuální režim a podle potřeby si zobrazit také doplňující historii informací. Ovládací prvky proto byly navrženy tak, aby byly snadno dostupné a přehledně oddělené od samotné vizualizace datové struktury. Tím je zajištěno, že uživatel může současně sledovat stav algoritmu i pohodlně pracovat s jeho ovládáním.

Na obrázku 3.5.2 je znázorněn příklad návrhu ovládacích prvků simulace v režimu syntaxe. Součástí rozhraní jsou tlačítka pro pohyb mezi jednotlivými kroky, tlačítko pro ukončení režimu a tlačítko pro zobrazení historie informací.



Obrázek 3.5.2: Příklad ovládacích prvků simulace v režimu syntaxe

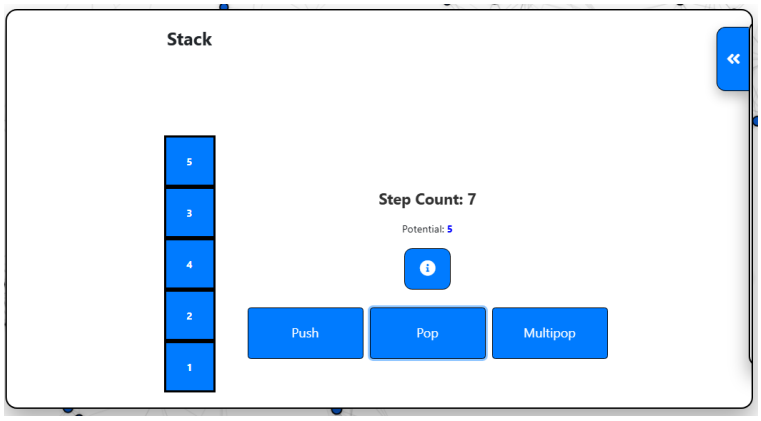
3.5.3 Návrh vizualizace datových struktur

Důležitou součástí návrhu uživatelského rozhraní bylo také zobrazení datových struktur používaných jednotlivými algoritmy. Cílem nebylo pouze zobrazit stav výpočtu formou číselných hodnot, ale navrhnout takovou vizualizaci, která uživateli umožní co nejlépe pochopit změny struktury v průběhu simulace. Z tohoto důvodu byla podoba vizualizace přizpůsobena konkrétnímu typu datové struktury, s níž daný algoritmus pracuje.

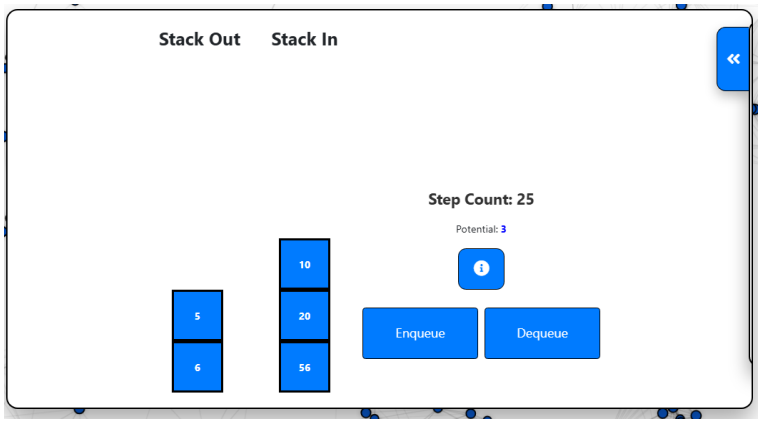
U algoritmu Multipop na zásobníku byla zvolena jednoduchá svislá reprezentace zásobníku, která přehledně ukazuje pořadí prvků i změny na vrcholu zásobníku. U fronty využívající dva zásobníky bylo naopak důležité oddělit oba zásobníky tak, aby bylo zřejmé, do které části se prvky vkládají a ze které se odebírají. U Splay stromu pak bylo nezbytné navrhnout stromovou vizualizaci, která umožní sledovat nejen obsah uzlů, ale i jejich vzájemné propojení a změny struktury při jednotlivých operacích.

Takto navržená vizualizace datových struktur podporuje názornost celé komponenty a umožňuje uživateli lépe porozumět tomu, jak se algoritmus chová při různých operacích. Uživatel tak nesleduje pouze výsledky výpočtu, ale i konkrétní změny uspořádání dat, které jsou pro pochopení amortizované složitosti důležité.

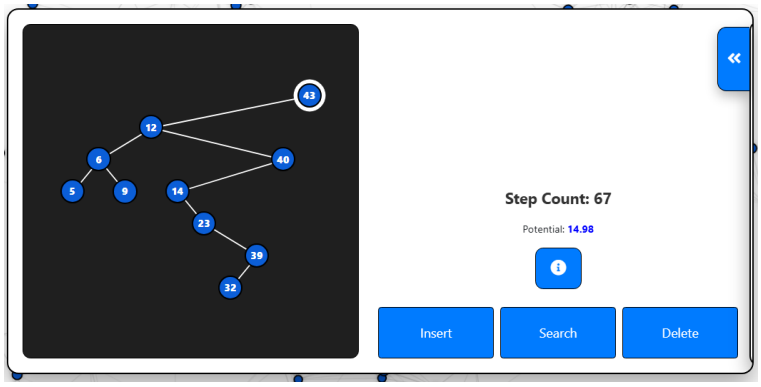
Na obrázku 3.5.3 jsou znázorněny příklady vizualizace datových struktur použitých v jednotlivých algoritmech, konkrétně zásobníku, fronty realizované pomocí dvou zásobníků a Splay stromu.



(a) Zásobník



(b) Fronta využívající dva zásobníky



(c) Splay strom

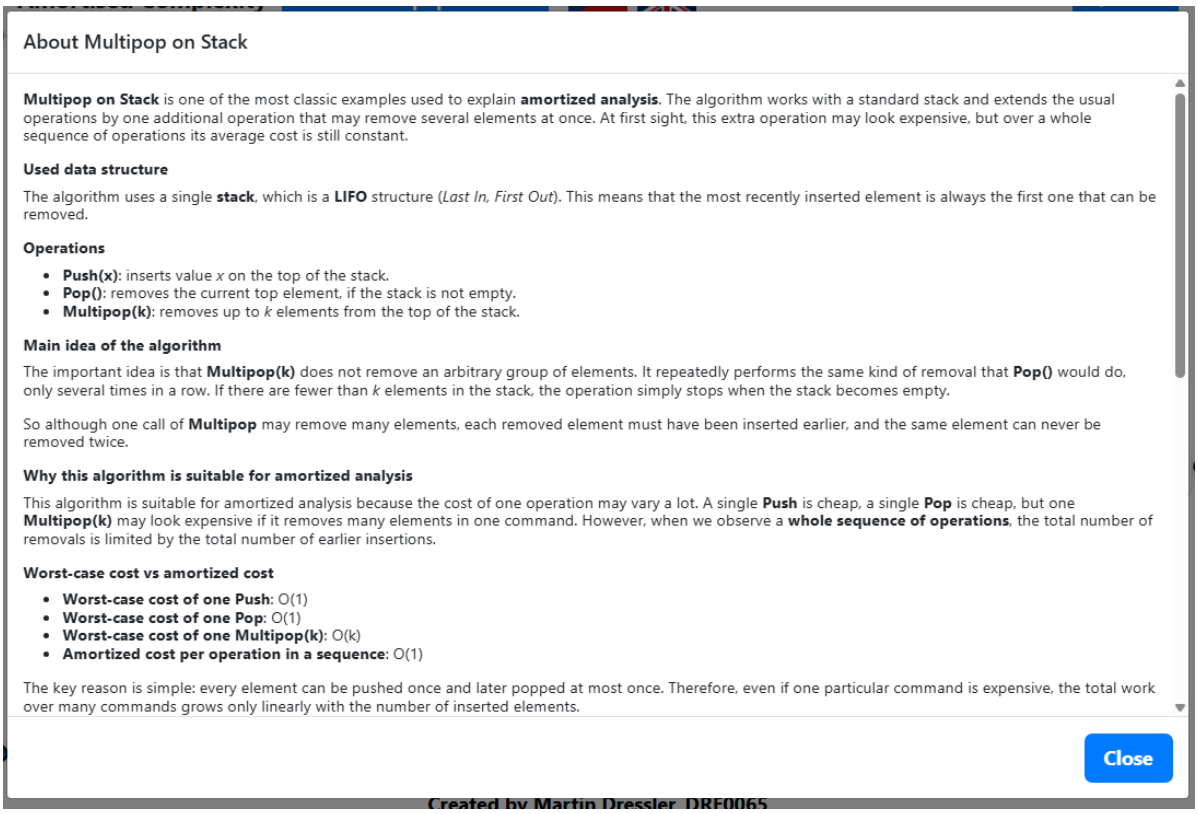
Obrázek 3.5.3: Příklady vizualizace datových struktur použitých v navrhované komponentě

3.5.4 Návrh informačních panelů a doprovodných vysvětlení o algoritmech

Součástí návrhu uživatelského rozhraní byly také informační panely a doprovodná vysvětlení vztahující se k jednotlivým algoritmům. Cílem těchto částí rozhraní nebylo pouze doplnit simulaci o další text, ale nabídnout uživateli přehledné vysvětlení základních vlastností algoritmu, použitých operací a jeho vztahu k amortizované analýze. Díky tomu může uživatel získat potřebný teoretický kontext ještě před samotnou prací se simulací nebo v jejím průběhu.

Návrh těchto informačních panelů vycházel z potřeby propojit praktickou ukázku s doprovodným výkladem. Uživatel tak nemá k dispozici pouze vizualizaci datové struktury a ovládací prvky simulace, ale také souvislý popis algoritmu, jeho hlavní myšlenky a vysvětlení, proč je daný algoritmus vhodný pro amortizovanou analýzu. Takové řešení podporuje lepší orientaci v problematice a současně posiluje výukový charakter celé komponenty.

Na obrázku 3.5.4 je znázorněn příklad informačního panelu věnovaného jednomu z vybraných algoritmů. Panel obsahuje základní popis algoritmu, přehled jeho operací, hlavní princip fungování i stručné vysvětlení jeho složitosti z hlediska amortizované analýzy.



Obrázek 3.5.4: Příklad informačního panelu s doprovodným vysvětlením vybraného algoritmu

3.6 Návrh struktury komponenty

Tato část se zaměřuje na návrh celkové struktury komponenty a na rozdělení jejích jednotlivých částí. Nejprve je popsáno oddělení společných prvků od částí specifických pro jednotlivé algoritmy. Následně je objasněna návaznost jednotlivých částí komponenty při práci uživatele. V závěru je pozornost věnována také možnostem budoucího rozšíření navrženého řešení.

3.6.1 Rozdělení komponenty na společnou a algoritmicky specifickou část

Celá struktura komponenty je rozdělena na prvky, které jsou společné pro všechny algoritmy, a na prvky, které jsou vytvořeny přímo pro konkrétní algoritmus. Mezi prvky patřící do společné části tvoří zejména hlavní stránka, výběr algoritmů, rozložení UI, obecné ovládací prvky a některé informační panely. Naopak část specifická pro daný algoritmus zahrnuje například samotnou logiku jednotlivých algoritmů, jejich konkrétní operace, způsob vizualizace příslušné datové struktury a doprovodné informace vztahující se k danému algoritmu. Díky tomuto rozdělení je možné zachovat jednotný vzhled a princip ovládání celé komponenty, ale zároveň ponechat dostatečný prostor pro přizpůsobení jednotlivým algoritmům.

3.6.2 Návaznost jednotlivých částí komponenty

Při návrhu byl brán zřetel také na návaznost jednotlivých částí z pohledu uživatele. Práce s komponentou začíná na hlavní stránce, kde si uživatel zvolí konkrétní algoritmus. Poté si vybere typ vstupu a sleduje průběh algoritmu, hodnoty důležité pro amortizovanou analýzu a doplňující vysvětlení. Jednotlivé části komponenty byly vytvořeny tak, aby na sebe logicky navazovaly a vytvářely ucelené prostředí pro práci uživatele.

3.6.3 Možnosti budoucího rozšíření

Při návrhu struktury bylo vhodné zohlednit také možnost budoucího rozšíření. Pokud by bylo potřeba aplikaci v budoucnu dále rozšířit, neměl by být problém přidat další algoritmy vhodné pro amortizovanou analýzu, aniž by bylo nutné zásadně měnit základní princip fungování celé komponenty. Stejným způsobem by bylo možné rozšiřovat informační panely, přidávat nové režimy práce se vstupy a další prvky.

Kapitola 4

Implementace komponenty

Kapitola 5

Závěr

Zdroje

1. SAWA, Zdeněk. *Teoretická informatika*. Dostupné také z: `https://www.cs.vsb.cz/sawa/ti/slides/ti-slides-03.pdf`. Podklady pro přednášky, VŠB – Technická univerzita Ostrava.
2. KOZEN, Dexter. *CS 3110 Lecture 20: Amortized Analysis*. Dostupné také z: `http://www.cs.cornell.edu/courses/cs3110/2011sp/lectures/lec20-amortized/amortized.htm`. Cornell University.
3. MASSACHUSETTS INSTITUTE OF TECHNOLOGY. *Big O notation*. Dostupné také z: `https://web.mit.edu/16.070/www/lecture/big_o.pdf`. Studijní materiál.
4. TARJAN, Robert Endre. Amortized Computational Complexity. *SIAM Journal on Algebraic and Discrete Methods*. 1985, roč. 6, č. 2, s. 306–318. Dostupné z DOI: `10.1137/0606031`.
5. CARNEGIE MELLON UNIVERSITY. *Amortized Analysis*. Dostupné také z: `https://www.cs.cmu.edu/~15451-f23/lectures/lecture05-amortized.pdf`. Lecture notes for 15-451: Algorithm Design and Analysis.
6. CORNELL UNIVERSITY. *Lecture 25: Amortized Analysis*. 2014. Dostupné také z: `https://www.cs.cornell.edu/courses/cs3110/2014fa/lectures/25/lec25.pdf`. CS 3110 lecture notes.
7. PUCELLA, Riccardo. *Notes on Programming Standard ML of New Jersey*. 2001. Dostupné také z: `https://www.cs.cornell.edu/riccardo/prog-smlnj/notes-011001.pdf`. Lecture notes.
8. SLEATOR, Daniel Dominic; TARJAN, Robert Endre. Self-Adjusting Binary Search Trees. *Journal of the ACM*. 1985, roč. 32, č. 3, s. 652–686. Dostupné také z: `https://www.cs.cmu.edu/~sleator/papers/self-adjusting.pdf`.